



UNIVERSITAT OBERTA DE CATALUNYA (UOC)
MÁSTER UNIVERSITARIO EN CIENCIA DE DATOS (*DATA SCIENCE*)

TRABAJO FINAL DE MÁSTER

ÁREA: REINFORCED LEARNING

Impacto de la complejidad de las observaciones en el rendimiento de algoritmos de Aprendizaje por Refuerzo en un entorno Pacman

Autor: Alejandro Suau Ruiz

Tutor: Marc Borrás Camarasa

Profesor: David Masip Rodó

Palma de Mallorca, 28 de diciembre de 2025

Créditos/Copyright

El presente Trabajo Final de Máster, incluyendo tanto la memoria escrita como el código fuente desarrollado, es una obra original del autor.

Licencia de la documentación

La memoria de este trabajo se distribuye bajo la licencia **Reconocimiento – NoComercial – CompartirIgual (CC BY-NC-SA 3.0 España)** de **Creative Commons**, que permite su copia, distribución y adaptación con fines no comerciales, siempre que se cite adecuadamente la autoría y se mantenga la misma licencia.

Licencia del código fuente

El código fuente desarrollado específicamente para este proyecto se distribuye bajo una licencia de software libre tipo **MIT**, que permite su uso, modificación y redistribución, con la única obligación de mantener el aviso de copyright original.

El código incluye exclusivamente desarrollo propio del autor, salvo las dependencias de terceros indicadas a continuación.

Uso de software y recursos de terceros

Este proyecto hace uso de bibliotecas y herramientas de terceros, todas ellas distribuidas bajo licencias de código abierto compatibles, entre las que destacan:

- **Python** y su ecosistema científico.
- **Gymnasium**, como interfaz estándar de entornos de Aprendizaje por Refuerzo.

- **Stable-Baselines3**, para la implementación de algoritmos DQN, A2C y PPO.
- **NumPy**, **Matplotlib** y **Pandas** para análisis y visualización de resultados.

Las marcas, nombres comerciales y logotipos mencionados pertenecen a sus respectivos propietarios y se utilizan únicamente con fines académicos y descriptivos.

Colaboraciones

No ha habido colaboradores externos en el desarrollo del presente trabajo. La totalidad del diseño experimental, implementación, ejecución de experimentos y redacción de la memoria han sido realizados por el autor.

FICHA DEL TRABAJO FINAL

Título del trabajo:	Impacto de la complejidad de las observaciones en el rendimiento
Nombre del autor:	Alejandro Suau Ruiz
Nombre del colaborador/a docente:	Marc Borrás Camarasa
Nombre del PRA:	David Masip Rodó
Fecha de entrega (mm/aaaa):	01/2026
Titulación o programa:	Máster Universitario de Data Science
Área del Trabajo Final:	Reinforced Learning
Idioma del trabajo:	Español
Palabras clave	Aprendizaje por refuerzo, Complejidad del estado, Pacman

Dedicatoria/Cita

A mi papá. A quién le hubiese gustado enormemente haberme visto completar el Máster.

Agradecimientos

Al tutor del TFM, Marc, por haberme acompañado durante todo el proceso y haber soportado mi ingente cantidad de mensajes y preguntas.

Abstract

This work investigates how the complexity of state representation affects the performance of reinforcement learning algorithms in a simplified Pacman environment. The central idea is that an agent’s success depends not only on the algorithm itself but also on the quality and richness of the information it receives from the environment. To analyze this, we designed a set of progressively more complex observation spaces, ranging from the basic positions of the player and the ghost to configurations that include the presence and duration of power-ups and the distribution of coins across quadrants. Experiments were carried out using widely adopted algorithms such as PPO, A2C, and DQN, implemented with the Stable-Baselines3 library. Controlled training with different random seeds and consistent performance metrics allowed us to compare both learning speed and the stability of the learned policies. The results show that increasing state complexity does not always lead to better performance: in some cases, the additional information introduces noise and hinders convergence. However, for specific configurations, the agent was able to exploit the extra information to achieve more robust behavior. In conclusion, the project highlights the importance of observation design in the success of reinforcement learning agents, stressing the need to balance simplicity and expressiveness depending on the task and the chosen algorithm.

Keywords: Reinforcement learning, State complexity, Pacman, Stable-Baselines3, Gymnasium

Resumen

Este trabajo explora cómo la complejidad de la representación del estado influye en el rendimiento de los algoritmos de aprendizaje por refuerzo en un entorno simplificado de Pacman. Partimos de la premisa de que un agente no aprende únicamente por el algoritmo empleado, sino también por la calidad y la riqueza de la información que percibe del entorno. Para analizarlo, hemos diseñado un conjunto de observaciones progresivamente más complejas, desde la posición básica de jugador y fantasma, hasta configuraciones que incluyen la presencia y duración de comodines o la distribución de monedas por cuadrantes. La experimentación se ha llevado a cabo con algoritmos ampliamente utilizados, como PPO, A2C y DQN, implementados con la librería Stable-Baselines3. Se han realizado entrenamientos controlados con semillas distintas y métricas de rendimiento homogéneas, lo que ha permitido comparar tanto la velocidad de aprendizaje como la estabilidad de las políticas aprendidas. Los resultados muestran que un aumento en la complejidad del estado no garantiza siempre un mejor desempeño: en ciertos casos, la mayor riqueza de información introduce ruido y dificulta la convergencia. Sin embargo, para determinadas configuraciones el agente logra aprovechar la información adicional y alcanzar un comportamiento más robusto. En conclusión, el proyecto confirma la relevancia del diseño de observaciones en el éxito de un agente de aprendizaje por refuerzo, subrayando la necesidad de equilibrar simplicidad y expresividad en función del objetivo y del algoritmo empleado.

Palabras clave: Aprendizaje por refuerzo, Complejidad del estado, Pacman, Stable-Baselines3, Gymnasium

Índice general

Abstract	vi
Resumen	vii
Índice	viii
Lista de Figuras	1
1. Introducción	2
1. Contexto y motivación	3
1.1. Justificación e interés	3
1.2. Motivación personal	4
2. Objetivos	5
2.1. Objetivo principal	5
2.2. Objetivos específicos	5
3. Sostenibilidad, diversidad y desafíos ético/sociales	5
4. Enfoque y metodología	6
4.1. Estrategias consideradas	6
4.2. Estrategia seleccionada	7
2. Estado del arte	8
1. Fundamentos del Aprendizaje por Refuerzo	8
2. Avances en Deep Reinforcement Learning	9
3. Síntesis y vacíos detectados	11
3. Planificación del proyecto	13
1. Recursos necesarios	13
2. Planificación temporal y tareas	13
3. Diagrama de Gantt simplificado	15
4. Hitos principales del proyecto	16

4. Metodología y diseño experimental	17
1. Visión general del proyecto	17
2. Infraestructura del proyecto y repositorio	18
3. Diseño del entorno <i>SimplePacmanEnv</i>	19
3.1. Descripción general del entorno	19
3.2. Motivación para implementar un entorno propio	21
3.3. Espacio de estados y observaciones	21
3.4. Espacio de acciones y dinámica del agente	23
3.5. Comportamiento del fantasma	24
3.6. Sistema de recompensas	25
3.7. Condiciones de terminación de los episodios	26
4. Algoritmos de aprendizaje por refuerzo utilizados	27
4.1. Configuración común de entrenamiento	27
4.2. Entrenamiento con Deep Q-Network (DQN)	28
4.3. Entrenamiento con Advantage Actor-Critic (A2C)	29
4.4. Entrenamiento con Proximal Policy Optimization (PPO)	30
5. Resultados experimentales	32
1. Métricas de evaluación y criterios de análisis	32
2. Resultados por algoritmo	33
2.1. Resultados con Deep Q-Network (DQN)	33
2.2. Resultados con Advantage Actor-Critic (A2C)	35
2.3. Resultados con Proximal Policy Optimization (PPO)	37
3. Comparación entre algoritmos	39
4. Impacto de la complejidad del estado	41
6. Conclusiones y trabajo futuro	44
1. Conclusiones del trabajo	44
2. Evaluación del grado de logro de los objetivos	45
3. Evaluación crítica de la planificación y metodología	45
4. Consideraciones de sostenibilidad, diversidad y aspectos ético-sociales	46
5. Trabajo futuro	46
Bibliografía	47

Índice de figuras

1.1. Imagen del juego original de Pac-man de 1980	4
4.1. Agente entrenado jugando una partida de simulación	20
5.1. Mean reward con algoritmo de DQN entrenado con 5M de steps	34
5.2. Completion ratio con algoritmo de DQN entrenado con 5M de steps	34
5.3. Std reward con algoritmo de DQN entrenado con 5M de steps	35
5.4. Completion ratio con algoritmo de A2C entrenado con 5M de steps	36
5.5. Completion ratio con algoritmo de A2C entrenado con 5M de steps	37
5.6. Std reward con algoritmo de A2C entrenado con 5M de steps	37
5.7. Completion ratio con algoritmo de PPO entrenado con 5M de steps	38
5.8. Mean reward con algoritmo de PPO entrenado con 5M de steps	38
5.9. Std reward con algoritmo de PPO entrenado con 5M de steps	39
5.10. Comparativa de mean reward entre algoritmos con 5M de steps	40
5.11. Comparativa de std reward entre algoritmos con 5M de steps	40
5.12. Comparativa de completion ratio entre algoritmos con 5M de steps	41

Capítulo 1

Introducción

El *Aprendizaje por Refuerzo* (*Reinforcement Learning*, RL) se ha consolidado como una de las ramas más activas y prometedoras del aprendizaje automático. A diferencia del aprendizaje supervisado, en el que el modelo aprende a partir de ejemplos etiquetados, el RL se fundamenta en la interacción continua de un agente con un entorno, buscando maximizar una recompensa acumulada. Este paradigma ha permitido resolver problemas complejos de toma de decisiones secuenciales en campos tan diversos como la robótica, los videojuegos, la gestión de recursos o la conducción autónoma.

En los últimos años, la combinación del RL con redes neuronales profundas —conocida como *Deep Reinforcement Learning* (DRL)— ha supuesto un gran avance, al permitir que los agentes aprendan directamente a partir de observaciones de alta dimensión. Algoritmos como *Deep Q-Network* (DQN) o *Proximal Policy Optimization* (PPO) han demostrado su eficacia en entornos complejos como Atari o MuJoCo, donde el espacio de estados y de acciones puede ser extremadamente amplio.

Sin embargo, el rendimiento de un agente no depende únicamente del algoritmo de aprendizaje empleado, sino también de la representación del estado con la que percibe el entorno. En otras palabras, la complejidad de la observación influye directamente en la capacidad del agente para generalizar, explorar y aprender políticas efectivas. Representaciones demasiado simples pueden limitar el aprendizaje al ocultar información relevante, mientras que representaciones demasiado complejas pueden ralentizar la convergencia y requerir una mayor capacidad de red o una exploración más prolongada.

En este Trabajo Fin de Máster se analiza precisamente cómo afecta la complejidad del estado observado al rendimiento del agente en un entorno controlado. Para ello, se ha desarrollado un entorno propio denominado *SimplePacmanEnv*, inspirado en el clásico juego *Pac-Man*, pero adaptado para experimentos de RL. Este entorno, implementado con *Gymnasium* y *Stable-Baselines3*, permite configurar diferentes niveles de complejidad en la observación —desde

representaciones mínimas (solo posiciones del jugador y el fantasma) hasta observaciones enriquecidas con información global sobre las monedas o incluso representaciones tipo imagen—, manteniendo constantes la dinámica del entorno y la estructura de recompensas.

Sobre esta base se han entrenado y evaluado tres algoritmos representativos de distintas familias del RL profundo: *Advantage Actor-Critic* (A2C), *Proximal Policy Optimization* (PPO) y *Deep Q-Network* (DQN). El objetivo es cuantificar cómo evoluciona el aprendizaje y el rendimiento final de cada agente en función de la información proporcionada por el entorno, y determinar si existe una relación clara entre la complejidad de la representación del estado y la eficiencia del aprendizaje.

Además, el proyecto incorpora herramientas complementarias como *TensorBoard* para la monitorización en tiempo real del entrenamiento, y el uso de *EvalCallback* para realizar evaluaciones periódicas y registrar automáticamente el mejor modelo alcanzado. Estos mecanismos no solo facilitan el análisis experimental, sino que aportan rigor y reproducibilidad a los resultados obtenidos.

En conjunto, este trabajo busca aportar una visión práctica y experimental sobre la influencia de la observación en entornos de aprendizaje por refuerzo, conectando los fundamentos teóricos con un estudio aplicado y reproducible, y poniendo de manifiesto la importancia del diseño de los estados en la efectividad de los algoritmos de RL.

1. Contexto y motivación

1.1. Justificación e interés

El estudio de la relación entre la complejidad del estado y el rendimiento del agente resulta relevante tanto desde el punto de vista teórico como práctico:

- Permite entender la eficiencia y capacidad de generalización de distintos algoritmos (A2C, PPO, DQN) en entornos de distinta dimensionalidad.
- Aporta evidencia empírica sobre el compromiso entre simplicidad del modelo y capacidad de representación, tema central en el diseño de entornos y arquitecturas RL.
- Facilita la comparación controlada entre configuraciones de observación, aislando el impacto del estado sin alterar dinámicas, recompensas ni mecánicas de juego.
- Además, el uso de un entorno propio y reproducible contribuye al valor educativo e investigativo, permitiendo futuras extensiones y análisis.

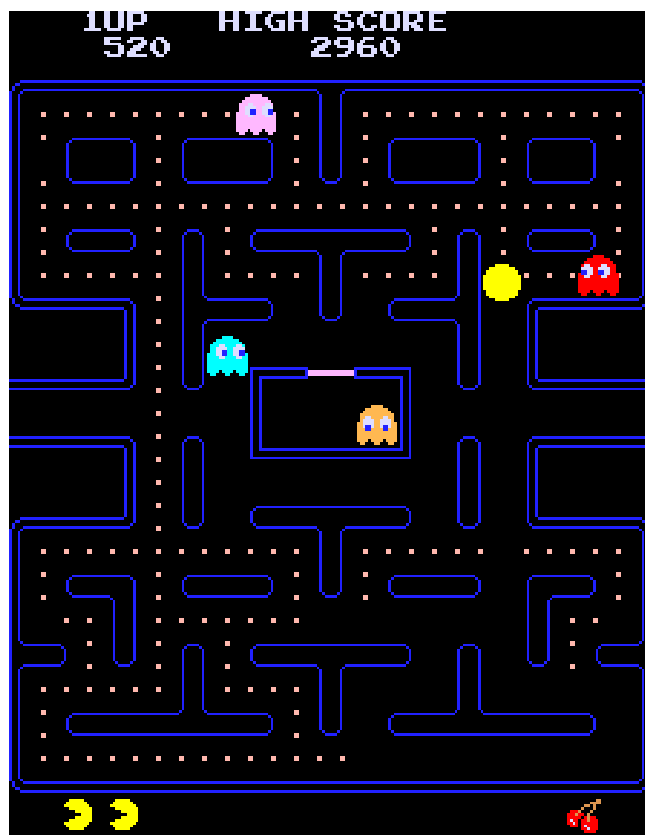


Figura 1.1: Imagen del juego original de Pac-man de 1980

1.2. Motivación personal

Mi motivación para realizar este proyecto es principalmente educativa e investigativa. Busco profundizar en el campo del Aprendizaje por Refuerzo, comprendiendo de forma práctica cómo los agentes aprenden a través de la interacción con su entorno y cómo la cantidad de información observada afecta dicho proceso.

El trabajo ha supuesto un ejercicio de integración de conocimientos adquiridos durante el Máster (aprendizaje automático, programación en Python, redes neuronales, y metodologías experimentales) con el objetivo de desarrollar una visión más completa y aplicada del RL. En última instancia, la meta ha sido explorar, aprender y comprender los fundamentos de la toma de decisiones secuenciales bajo incertidumbre.

2. Objetivos

2.1. Objetivo principal

Evaluar cómo el nivel de complejidad del espacio de observación afecta el rendimiento de distintos algoritmos de Aprendizaje por Refuerzo, manteniendo constantes el entorno, las recompensas y la dinámica del juego.

2.2. Objetivos específicos

1. Diseñar e implementar un entorno controlado y parametrizable (basado en Pac-Man) que permita modificar el tipo de observación percibida por el agente.
2. Entrenar y comparar varios algoritmos de RL (A2C, PPO, DQN) bajo distintos modos de observación.
3. Analizar métricas de rendimiento (recompensa media, estabilidad, convergencia, tiempo de entrenamiento).
4. Evaluar la relación entre dimensionalidad del estado y eficiencia del aprendizaje.
5. Documentar la metodología y resultados, destacando las implicaciones para futuros desarrollos o estudios.

3. Sostenibilidad, diversidad y desafíos ético/sociales

A continuación se valoran los posibles impactos en materia de sostenibilidad, ética y diversidad.

Sostenibilidad. El trabajo se ha realizado íntegramente en un entorno digital y no requiere recursos materiales adicionales, por lo que su impacto medioambiental directo es muy reducido. El único consumo relevante proviene del uso de recursos computacionales durante el entrenamiento de los agentes. Se ha procurado minimizar este impacto mediante el uso de modelos ligeros y tiempos de entrenamiento moderados, evitando configuraciones innecesariamente costosas. Desde una perspectiva positiva, la metodología promueve la eficiencia en la experimentación mediante la reutilización de entornos y scripts reproducibles, contribuyendo así a un uso más sostenible de los recursos de investigación. El proyecto no se ve afectado por ninguna normativa específica en materia medioambiental ni tiene relación directa con los Objetivos de Desarrollo Sostenible (ODS), más allá de fomentar el desarrollo de tecnologías digitales eficientes y abiertas.

Comportamiento ético y responsabilidad social. El proyecto no involucra datos personales, información sensible ni toma de decisiones sobre personas o colectivos. Su propósito es exclusivamente académico y experimental. Se han seguido principios de transparencia y reproducibilidad, haciendo uso de bibliotecas abiertas como *Stable-Baselines3* y *Gymnasium*, lo que garantiza la trazabilidad del código y los resultados. En este sentido, el trabajo se alinea con los principios deontológicos de la ingeniería informática, promoviendo la investigación responsable y el acceso abierto al conocimiento. No se identifican impactos negativos sobre el empleo ni sobre aspectos legales o de seguridad.

Diversidad, género y derechos humanos. El contenido técnico del trabajo no tiene incidencia directa en cuestiones de género, diversidad o derechos humanos. Sin embargo, se reconoce la importancia de la diversidad en la investigación en inteligencia artificial y el compromiso con un lenguaje inclusivo y accesible en la documentación y la memoria del proyecto. Asimismo, el entorno desarrollado puede emplearse como herramienta docente o de experimentación, favoreciendo la accesibilidad y la igualdad de oportunidades en la formación en técnicas de aprendizaje por refuerzo.

En conjunto, el proyecto presenta un impacto ético, social y medioambiental neutro o ligeramente positivo, destacando por su orientación a la investigación abierta, la eficiencia computacional y el cumplimiento de buenas prácticas profesionales en el ámbito de la inteligencia artificial.

4. Enfoque y metodología

El presente trabajo adopta un enfoque experimental y comparativo orientado a analizar la influencia de la complejidad del estado sobre el rendimiento de los algoritmos de Aprendizaje por Refuerzo (RL).

4.1. Estrategias consideradas

Durante la fase inicial del proyecto se evaluaron tres posibles enfoques:

1. **Uso de entornos estándar (Gym / Atari):** Permitiría aprovechar entornos consolidados como CartPole o Breakout. Sin embargo, estos no ofrecen un control granular sobre la estructura del estado, lo que impide aislar experimentalmente el efecto de la observación.
2. **Simulación matemática abstracta (MDP sintético):** Un enfoque más teórico, usando representaciones vectoriales arbitrarias para estudiar el tamaño del espacio de estado.

Aunque más simple de implementar, carece de un contexto visual y de dinámica tangible que facilite la interpretación y comparación intuitiva de los resultados.

3. **Diseño de un entorno personalizado (Pacman simplificado):** Este enfoque combina realismo y control. Permite modificar el nivel de información observable manteniendo constantes las reglas y recompensas. Además, el dominio es familiar y visualmente interpretable, lo que facilita tanto la depuración como la comunicación de los resultados.

4.2. Estrategia seleccionada

Se eligió la **tercera estrategia**, basada en el desarrollo de un entorno propio —SimplePacmanEnv— compatible con Gymnasium y Stable-Baselines3. Esta elección resulta la más adecuada por las siguientes razones:

- Permite **aislar experimentalmente** la variable de interés (la complejidad del estado), manteniendo el resto de factores constantes.
- Facilita la **comparación justa** entre distintos algoritmos de RL bajo las mismas condiciones.
- Favorece la **reproducibilidad y extensibilidad** del experimento (se pueden añadir observaciones o cambiar reglas sin alterar la estructura general).
- Proporciona una **base educativa sólida**, al exigir una comprensión integral del pipeline de RL: diseño de entorno, configuración del agente y análisis de resultados.

Capítulo 2

Estado del arte

1. Fundamentos del Aprendizaje por Refuerzo

El *Aprendizaje por Refuerzo* (Reinforcement Learning, RL) constituye una de las ramas más relevantes del aprendizaje automático orientadas a la toma de decisiones secuenciales bajo incertidumbre. En este paradigma, un agente interactúa con un entorno mediante un proceso de prueba y error, buscando maximizar una recompensa acumulada a largo plazo. Formalmente, este proceso se modela como un *Proceso de Decisión de Markov* (MDP), definido por el conjunto (S, A, P, R, γ) , donde S representa el espacio de estados, A el conjunto de acciones posibles, $P(s'|s, a)$ la función de transición de estados, $R(s, a)$ la función de recompensa y $\gamma \in [0, 1]$ el factor de descuento que pondera la importancia de las recompensas futuras [Sutton and Barto, 2018].

El objetivo de un agente es encontrar una política óptima $\pi^*(a|s)$ que maximice el retorno esperado:

$$G_t = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right], \quad (2.1)$$

donde r_{t+k+1} representa la recompensa obtenida k pasos después del tiempo t . A diferencia del aprendizaje supervisado, el RL no dispone de un conjunto fijo de ejemplos etiquetados: el conocimiento se adquiere mediante la experiencia directa con el entorno.

Los algoritmos de RL pueden clasificarse en dos grandes familias. Los **métodos basados en valor** estiman la utilidad esperada de los estados o pares estado-acción a través de funciones $V(s)$ o $Q(s, a)$, como en los algoritmos Q-Learning o SARSA. Por otro lado, los **métodos basados en política** parametrizan directamente la política $\pi_\theta(a|s)$ mediante un modelo (típicamente una red neuronal) y optimizan sus parámetros θ a partir del gradiente de la recompensa esperada. La combinación de ambos enfoques da lugar a los métodos **actor-crítico**, donde un componente

(*crítico*) estima el valor de los estados y otro (*actor*) actualiza la política en función de dicha estimación.

En este marco, la **definición del estado** s_t es un factor determinante. La información que el agente percibe condiciona su capacidad para generalizar, explorar y aprender políticas efectivas. Una representación excesivamente simple puede ocultar detalles relevantes del entorno, mientras que una observación demasiado compleja puede introducir ruido, ralentizar la convergencia y requerir redes de mayor capacidad. Por ello, la **complejidad del estado observado** constituye una variable crítica para la eficiencia del aprendizaje, y su análisis es el eje central del presente trabajo.

2. Avances en Deep Reinforcement Learning

La integración del aprendizaje por refuerzo con redes neuronales profundas dio lugar al *Aprendizaje por Refuerzo Profundo* (Deep Reinforcement Learning, DRL), que permite aproximar funciones de valor o políticas directamente a partir de observaciones de alta dimensión, como imágenes o secuencias temporales. Este paradigma ha sido fundamental para extender el RL a entornos complejos donde las representaciones manuales del estado resultan inviables [Mnih et al., 2015].

Deep Q-Network (DQN)

El punto de inflexión en la historia del DRL se produjo con el algoritmo **Deep Q-Network (DQN)** desarrollado por Mnih et al. [2015], que combinó *Q-Learning* con redes neuronales convolucionales. DQN aprende una función de acción-valor $Q(s, a; \theta)$ que estima la recompensa esperada al ejecutar una acción a en un estado s , siguiendo una política π derivada de dicha estimación (normalmente, una política ϵ -greedy). El objetivo de la red es minimizar la diferencia entre la predicción actual y el valor objetivo definido por la ecuación de Bellman:

$$L(\theta) = \mathbb{E} \left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2 \right], \quad (2.2)$$

donde θ^- son los parámetros de una red objetivo (*target network*) que se actualiza de forma retardada para estabilizar el aprendizaje. El uso de una memoria de experiencias (*replay buffer*) permite muestrear de manera aleatoria transiciones (s, a, r, s') , reduciendo la correlación temporal entre los datos.

Gracias a estos mecanismos, DQN consiguió por primera vez que un agente alcanzara rendimientos comparables a los humanos en múltiples videojuegos de Atari, aprendiendo directamente a partir de píxeles sin información previa sobre las reglas del entorno.

A partir de este modelo original surgieron numerosas variantes orientadas a mejorar la estabilidad y eficiencia, como *Double DQN*, que corrige el sesgo optimista de la estimación del valor máximo, o *Dueling DQN*, que separa la estimación del valor de estado y la ventaja de cada acción.

Advantage Actor-Critic (A2C)

A diferencia de DQN, los métodos basados exclusivamente en valor no se adaptan bien a entornos con espacios de acción continuos. Para superar esta limitación surgieron los **métodos actor-crítico**, que combinan una *red actor*, encargada de representar la política $\pi_\theta(a|s)$, y una *red crítico*, que estima la función de valor $V_\phi(s)$ y evalúa la calidad de las acciones seleccionadas. El algoritmo **Advantage Actor-Critic (A2C)**, versión síncrona del A3C (*Asynchronous Advantage Actor-Critic*) [Mnih et al., 2016], ejecuta múltiples agentes en paralelo que comparten la misma política y sincronizan sus actualizaciones periódicamente.

El crítico proporciona una señal de error (*advantage*) definida como:

$$A(s, a) = Q(s, a) - V(s), \quad (2.3)$$

que mide cuánto mejor resulta ejecutar la acción a frente al valor medio esperado del estado. El actor ajusta su política en la dirección del gradiente del rendimiento esperado:

$$\nabla_\theta J(\theta) = \mathbb{E}[A(s, a) \nabla_\theta \log \pi_\theta(a|s)]. \quad (2.4)$$

Este esquema reduce la varianza del gradiente respecto a los métodos puramente basados en política, logrando un equilibrio entre estabilidad y eficiencia de aprendizaje.

Proximal Policy Optimization (PPO)

Posteriormente, Schulman et al. [2017] introdujeron el algoritmo **Proximal Policy Optimization (PPO)**, que mejoró la estabilidad del entrenamiento mediante una actualización de política acotada. En lugar de aplicar grandes cambios en la política tras cada actualización, PPO optimiza una función de pérdida que penaliza desviaciones excesivas entre la política nueva π_θ y la antigua $\pi_{\theta_{\text{old}}}$:

$$L^{CLIP}(\theta) = \mathbb{E}[\text{mín}(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)], \quad (2.5)$$

donde $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ representa la proporción entre políticas. El operador *clip* limita la magnitud del cambio a un rango controlado $(1 \pm \epsilon)$, asegurando una mejora progresiva

sin sobreajustes bruscos. Este mecanismo, junto con el uso de minibatches y actualizaciones múltiples por lote, convierte a PPO en un algoritmo especialmente robusto y fácil de entrenar, siendo actualmente uno de los más utilizados en investigación y aplicaciones prácticas.

Síntesis comparativa

En síntesis, DQN, A2C y PPO representan tres paradigmas complementarios del Aprendizaje por Refuerzo Profundo:

- **DQN:** aprendizaje basado en valor, idóneo para entornos con acciones discretas; su estabilidad depende fuertemente de la correlación entre muestras y de la calidad de la observación.
- **A2C:** enfoque actor-crítico que combina estimación de valor y política, reduciendo la varianza del gradiente y favoreciendo el aprendizaje paralelo.
- **PPO:** optimización de políticas proximales que introduce una restricción explícita sobre el cambio de política, logrando robustez y generalización en entornos complejos.

Estas características los convierten en candidatos idóneos para analizar de forma comparativa cómo la **complejidad de la observación** afecta el rendimiento del aprendizaje, manteniendo constante la dinámica y la recompensa del entorno.

3. Síntesis y vacíos detectados

La revisión de la literatura evidencia que, aunque el Aprendizaje por Refuerzo Profundo ha experimentado avances notables en la última década, gran parte de los esfuerzos se ha centrado en la **optimización de los algoritmos** —ya sea mediante arquitecturas, estrategias de exploración o funciones de pérdida—, mientras que el impacto de la representación del estado ha recibido menor atención. En la mayoría de los estudios, el espacio de observación se trata como una variable fija del entorno, sin evaluar de manera sistemática cómo su complejidad o riqueza informacional influye en la eficacia del aprendizaje.

En respuesta a esta limitación, han surgido líneas de investigación como el *State Representation Learning* (SRL), que busca aprender representaciones latentes más compactas y relevantes para la toma de decisiones [Lesort et al., 2018, Ha and Schmidhuber, 2018, Hafner et al., 2020]. Estos trabajos utilizan modelos generativos o redes autoencoder para capturar información útil en espacios de menor dimensión, lo que facilita la generalización y reduce el coste computacional. No obstante, la mayoría de enfoques SRL se orientan a la **construcción automática de**

representaciones y no al estudio explícito del efecto de distintos niveles de complejidad observacional sobre el proceso de aprendizaje.

De forma complementaria, los estudios sobre entornos parcialmente observables [Hausknecht and Stone, 2015] han puesto de manifiesto cómo la falta de información puede limitar la convergencia y provocar políticas subóptimas. Sin embargo, apenas existen análisis controlados sobre el efecto contrario: cómo el exceso de información o la redundancia en el estado pueden degradar el rendimiento o ralentizar la convergencia.

Los entornos más utilizados como *benchmarks*, tales como Atari o MuJoCo, ofrecen espacios de observación fijos, lo que impide aislar experimentalmente esta variable. En años recientes, entornos modulares como *MiniGrid* [Chevalier-Boisvert et al., 2018] o *ProcGen* [Cobbe et al., 2020] han permitido cierta variabilidad estructural, aunque su propósito principal ha sido evaluar la capacidad de generalización entre tareas, no la complejidad del estado como tal.

En este contexto, se identifica un vacío experimental relevante: la ausencia de estudios que analicen de forma controlada la relación entre la **complejidad del espacio de observación** y el rendimiento de distintos algoritmos de RL manteniendo constantes las dinámicas, recompensas y reglas del entorno.

El presente trabajo busca contribuir a llenar ese vacío mediante el desarrollo de un entorno propio, *SimplePacmanEnv*, inspirado en el clásico juego Pac-Man. Dicho entorno permite parametrizar el nivel de información observable, desde representaciones mínimas (posición del agente y del fantasma) hasta configuraciones enriquecidas con comodines, distribución de monedas o vistas tipo imagen. Esta configuración experimental posibilita evaluar empíricamente cómo la cantidad y naturaleza de la información percibida influyen en la estabilidad del entrenamiento, la velocidad de convergencia y la calidad final de la política aprendida.

En conjunto, este análisis pretende aportar una visión empírica y reproducible sobre el equilibrio entre **simplicidad y expresividad** en el diseño de observaciones, contribuyendo así a una comprensión más profunda del papel de la percepción en el rendimiento de los agentes de aprendizaje por refuerzo.

Capítulo 3

Planificación del proyecto

1. Recursos necesarios

A continuación se detallan los recursos técnicos, de software y humanos requeridos para el desarrollo del proyecto. La selección de estos recursos se ha realizado considerando la naturaleza computacional del trabajo y la necesidad de garantizar reproducibilidad y eficiencia durante el proceso experimental.

Tipo de recurso	Descripción
Hardware	Ordenador personal con CPU multinúcleo (Intel i7 o equivalente), 16 GB de RAM y GPU NVIDIA opcional para acelerar el entrenamiento de redes neuronales.
Software	Sistema operativo Linux/Windows, Python 3.10+, librerías <i>Gymnasium</i> , <i>Stable-Baselines3</i> , <i>NumPy</i> , <i>Matplotlib</i> y <i>TensorBoard</i> para la monitorización de entrenamientos.
Control de versiones	Repositorio privado en GitHub para gestionar código, scripts y documentación.
Datos y experimentos	No se requieren datasets externos: los datos se generan de forma simulada a través del entorno SimplePacmanEnv .
Apoyo académico	Tutorías con el tutor del TFM y revisión periódica según las fases definidas en las PEC.

2. Planificación temporal y tareas

El proyecto se organiza siguiendo las fases de los módulos del TFM (de M1 a M5), que marcan los hitos principales de evaluación continua. Cada fase combina tareas de análisis,

desarrollo y documentación.

Fase / Módulo	Periodo aproximado	Descripción de tareas principales	Hito asociado
M1 – Definición y planificación del TFM	25 sep – 12 oct 2025	■ Definir objetivos y alcance del proyecto. ■ Revisar la viabilidad técnica y ética. ■ Elaborar propuesta inicial y planificación.	Entrega M1 (12 oct)
M2 – Estado del arte y fundamentos teóricos	13 oct – 2 nov 2025	■ Revisión bibliográfica sobre aprendizaje por refuerzo, entornos Gym y algoritmos A2C, PPO y DQN. ■ Identificación de trabajos similares y justificación del enfoque experimental.	Entrega M2 (2 nov)
M3 – Diseño e implementación del sistema	3 nov – 14 dic 2025	■ Implementar el entorno SimplePacmanEnv. ■ Desarrollar scripts de entrenamiento (<code>train_a2c.py</code>, <code>train_ppo.py</code>, <code>train_dqn.py</code>). ■ Validar funcionalidad y reproducibilidad. ■ Documentar el código.	Entrega M3 (14 dic)

M4 – Redacción y análisis de resultados	22 dic – 28 dic 2025	■ Entrenar los modelos con distintas configuraciones de observación. ■ Analizar resultados y elaborar gráficas comparativas. ■ Redactar la memoria y preparar la presentación audiovisual.	Entrega M4 (21–28 dic); vídeo: 6 ene 2026
M5 – Defensa y cierre del proyecto	9 ene – 30 ene 2026	■ Entrega final de la documentación al tribunal. ■ Presentación y defensa pública del trabajo. ■ Revisión y cierre de la memoria.	Entrega y defensa (9–30 ene)

3. Diagrama de Gantt simplificado

El diagrama de Gantt que se presenta a continuación resume de forma visual la distribución temporal de las actividades y su solapamiento a lo largo del semestre académico. Cada marca indica aproximadamente una semana de dedicación dentro del mes correspondiente.

Actividad / Mes	Sep	Oct	Nov	Dic	Ene	Feb
Definición y planificación (M1)	XX	X				
Estado del arte (M2)		XXX				
Diseño e implementación (M3)		X	XXXX	X		
Análisis y redacción (M4)				XXX	X	
Defensa y cierre (M5)					XXX	X

Nota: cada “X” representa aproximadamente una semana de trabajo dentro del mes correspondiente.

4. Hitos principales del proyecto

Finalmente, se resumen los hitos más relevantes del proyecto, vinculados a las Pruebas de Evaluación Continua (PEC) y a las entregas oficiales del calendario académico. Estos hitos marcan los puntos de control que estructuran el avance del trabajo desde su definición inicial hasta la defensa final.

Hito	Fecha aproximada	Descripción
H1 – Definición del TFM (PEC1)	12 oct 2025	Entrega del documento de definición y planificación del TFM.
H2 – Estado del arte (PEC2)	2 nov 2025	Entrega del marco teórico y análisis del contexto del trabajo.
H3 – Implementación (PEC3)	14 dic 2025	Finalización del entorno funcional y de los scripts de entrenamiento.
H4 – Redacción preliminar y final (PEC4)	21–28 dic 2025	Entrega del documento completo del TFM y del vídeo de presentación.
H5 – Defensa final (PEC5)	30 ene 2026	Presentación pública y defensa del trabajo ante el tribunal.

Capítulo 4

Metodología y diseño experimental

1. Visión general del proyecto

El objetivo principal de este trabajo es analizar de forma empírica cómo la complejidad del estado observado influye en el rendimiento de distintos algoritmos de Aprendizaje por Refuerzo. Para ello, se ha diseñado un marco experimental completo que abarca desde la creación de un entorno propio hasta la ejecución sistemática de entrenamientos, la optimización de hiperparámetros y la evaluación cuantitativa de los resultados obtenidos.

El proyecto se ha desarrollado siguiendo un enfoque modular y reproducible, apoyado en un repositorio de código que centraliza todos los componentes necesarios para la experimentación. Este repositorio incluye la definición del entorno, los scripts de entrenamiento para cada algoritmo, los procedimientos de evaluación, las herramientas de ajuste automático de hiperparámetros y los mecanismos de visualización del aprendizaje. Esta organización permite aislar responsabilidades, facilitar la replicabilidad de los experimentos y garantizar la trazabilidad de los resultados.

En primer lugar, se ha implementado desde cero un entorno de aprendizaje por refuerzo denominado *SimplePacmanEnv*, inspirado en el clásico videojuego Pac-Man. Este entorno ha sido diseñado con el objetivo de permitir un control preciso sobre la información observacional proporcionada al agente, manteniendo constantes las dinámicas del juego y la función de recompensa. De este modo, es posible evaluar de manera aislada el impacto de diferentes configuraciones de estado sobre el proceso de aprendizaje.

Sobre este entorno se han entrenado agentes utilizando tres algoritmos representativos del Aprendizaje por Refuerzo Profundo: DQN, A2C y PPO. Cada algoritmo se ha entrenado mediante un script específico, garantizando una configuración homogénea del entorno y del protocolo de evaluación. Inicialmente, se llevaron a cabo múltiples ejecuciones exploratorias con ajustes manuales de hiperparámetros, lo que permitió adquirir una comprensión práctica del comportamiento de los algoritmos y de la sensibilidad del aprendizaje a distintos parámetros.

Posteriormente, y con el fin de sistematizar este proceso, se incorporó la librería Optuna para realizar la optimización automática de hiperparámetros. Esta herramienta permitió explorar de manera eficiente espacios de búsqueda amplios, reduciendo el sesgo introducido por la selección manual y mejorando la calidad de los modelos entrenados. Dada la elevada carga computacional asociada a estos experimentos, se definieron estrategias de compromiso entre exhaustividad y coste temporal, incluyendo criterios de parada temprana y frecuencias de evaluación ajustadas.

El rendimiento de los agentes entrenados se evaluó mediante un protocolo común basado en múltiples episodios de prueba, utilizando métricas cuantitativas como la recompensa media, la duración de los episodios y tasas de éxito. Adicionalmente, se emplearon herramientas de monitorización como TensorBoard para visualizar el progreso del aprendizaje y analizar de forma cualitativa el comportamiento de los agentes durante el entrenamiento.

En conjunto, este enfoque metodológico permite abordar de manera estructurada la pregunta de investigación planteada, proporcionando un marco experimental sólido para estudiar la relación entre la complejidad del estado y el rendimiento de los algoritmos de Aprendizaje por Refuerzo.

2. Infraestructura del proyecto y repositorio

Con el objetivo de garantizar la reproducibilidad, organización y trazabilidad de los experimentos, todo el desarrollo del proyecto se ha centralizado en un repositorio de código versionado. Este repositorio constituye la base de la infraestructura experimental y contiene tanto la implementación del entorno como los scripts necesarios para el entrenamiento, la evaluación y el análisis de los agentes.

La estructura del proyecto sigue un enfoque modular, separando claramente las distintas responsabilidades funcionales. Esta organización facilita la extensión del trabajo, la repetición de experimentos y la comprensión del flujo completo por parte de terceros. De forma general, el repositorio se divide en los siguientes componentes principales:

- **Definición del entorno:** incluye la implementación completa del entorno *SimplePacmanEnv*, así como los ficheros auxiliares que definen constantes, recompensas y configuraciones comunes.
- **Entrenamiento de agentes:** contiene scripts independientes para el entrenamiento de cada algoritmo de aprendizaje por refuerzo considerado (DQN, A2C y PPO), permitiendo una ejecución controlada y repetible de los experimentos.
- **Optimización de hiperparámetros:** agrupa los scripts dedicados a la búsqueda automática de hiperparámetros mediante la librería Optuna, incluyendo la definición de los

espacios de búsqueda y los criterios de evaluación.

- **Evaluación y análisis:** recoge los procedimientos utilizados para evaluar los modelos entrenados bajo un protocolo común, así como la recopilación de métricas cuantitativas.
- **Visualización y ejecución interactiva:** incluye utilidades para observar el comportamiento del agente entrenado en el entorno, facilitando un análisis cualitativo del aprendizaje.

Esta separación permite modificar o ampliar cualquiera de los componentes sin afectar al resto del sistema. Por ejemplo, es posible introducir un nuevo algoritmo de aprendizaje o una nueva configuración de observación sin alterar el entorno base ni el protocolo de evaluación.

Adicionalmente, el proyecto hace uso de herramientas estándar del ecosistema de aprendizaje automático en Python, como *Gymnasium* para la definición del entorno y *Stable-Baselines3* para la implementación de los algoritmos de aprendizaje por refuerzo. La monitorización del entrenamiento se realiza mediante *TensorBoard*, lo que permite registrar y visualizar métricas clave como la recompensa acumulada, la longitud de los episodios o la evolución de la función de pérdida a lo largo del tiempo.

El uso de un repositorio estructurado y de herramientas ampliamente adoptadas contribuye a que los experimentos realizados en este trabajo puedan ser reproducidos y validados, alineándose con las buenas prácticas actuales en investigación experimental en Aprendizaje por Refuerzo.

3. Diseño del entorno *SimplePacmanEnv*

3.1. Descripción general del entorno

El entorno experimental desarrollado en este trabajo, denominado *SimplePacmanEnv*, está inspirado en el clásico videojuego *Pac-Man*. Su diseño responde a la necesidad de disponer de un entorno controlado que permita estudiar de forma aislada el impacto de la complejidad del estado observado sobre el rendimiento de los algoritmos de Aprendizaje por Refuerzo, manteniendo constantes las dinámicas del juego y la función de recompensa.

El entorno se ha implementado desde cero siguiendo la interfaz estándar de *Gymnasium*, lo que garantiza su compatibilidad directa con librerías de Aprendizaje por Refuerzo como *Stable-Baselines3*. En cada episodio, un agente (Pacman) se desplaza por una rejilla bidimensional compuesta por celdas que pueden representar muros, espacios vacíos o elementos coleccionables. El objetivo del agente es maximizar la recompensa acumulada mediante la recolección de monedas, evitando al mismo tiempo la colisión con un enemigo (fantasma).

A diferencia del videojuego original, el entorno ha sido deliberadamente simplificado para eliminar elementos que no son relevantes para el objetivo experimental, como múltiples enemigos,

laberintos dinámicos o comportamientos estocásticos complejos. Esta simplificación permite centrar el análisis en el efecto de la información observacional sin introducir factores de confusión adicionales.

El entorno presenta las siguientes características generales:

- **Espacio discreto y finito:** el mapa del entorno se define como una rejilla bidimensional de tamaño fijo, donde cada celda tiene un significado semántico claro (muro, espacio vacío o moneda).
- **Agente único y enemigo único:** el episodio involucra un solo agente controlado por el algoritmo de aprendizaje y un único fantasma con un comportamiento predefinido.
- **Acciones discretas:** el agente puede ejecutar un conjunto reducido de acciones, típicamente movimientos en las cuatro direcciones cardinales.
- **Episodios acotados:** cada episodio finaliza al cumplirse determinadas condiciones, como la recolección de todas las monedas, la colisión con el fantasma o la superación de un número máximo de pasos.

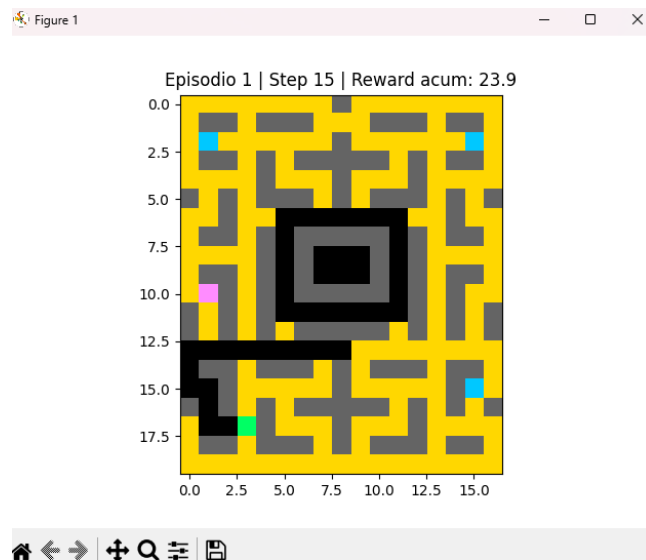


Figura 4.1: Agente entrenado jugando una partida de simulación

Uno de los aspectos diferenciales del entorno *SimplePacmanEnv* es la posibilidad de definir múltiples modos de observación. En función de la configuración seleccionada, el agente puede percibir desde una representación mínima del estado —por ejemplo, la posición relativa del agente y del fantasma— hasta observaciones más complejas que incluyen información adicional sobre el entorno, como la presencia de comodines, temporizadores o la distribución espacial de

monedas. Esta flexibilidad convierte al entorno en una herramienta idónea para estudiar de forma sistemática la relación entre la complejidad del estado y el comportamiento del aprendizaje.

En los apartados siguientes se describen con mayor detalle las dinámicas del entorno, el comportamiento de los distintos elementos y la definición formal de los espacios de estados, acciones y recompensas.

3.2. Motivación para implementar un entorno propio

Una contribución clave de este trabajo es la implementación completa desde cero de un entorno tipo Pac-Man (*SimplePacmanEnv*) en lugar de reutilizar entornos existentes. Aunque existen *benchmarks* ampliamente utilizados en Aprendizaje por Refuerzo (por ejemplo, Atari o MiniGrid), en la mayoría de ellos el espacio de observaciones viene fijado por el propio entorno y no permite modificar de manera controlada el nivel de información accesible al agente sin alterar simultáneamente otros factores (dinámica, recompensas o reglas).

Dado que la pregunta de investigación de este TFM se centra en el efecto de la complejidad del estado observado, era imprescindible disponer de un entorno donde pudiera parametrizarse de forma explícita qué información recibe el agente, manteniendo constantes el resto de componentes del MDP. Esto motivó el desarrollo de un entorno propio, compatible con Gymnasium y Stable-Baselines3, que permite definir múltiples modos de observación (mínimo, comodín binario, temporizador y agregación espacial de monedas) sin modificar la dinámica del juego ni el sistema de recompensas.

Esta capacidad de control experimental no es trivial: requiere implementar desde cero la simulación del entorno (rejilla, colisiones, transiciones, recompensas, terminación de episodios), así como asegurar el cumplimiento estricto de la interfaz de Gymnasium para permitir entrenamientos reproducibles con librerías estándar. En consecuencia, el entorno desarrollado actúa como un *benchmark* modular orientado específicamente a estudiar el compromiso entre simplicidad y expresividad en el diseño del estado.

3.3. Espacio de estados y observaciones

En el contexto del Aprendizaje por Refuerzo, el *estado* representa la información que el agente utiliza para decidir su acción en cada paso temporal. En el entorno *SimplePacmanEnv*, se distingue de manera explícita entre el estado completo del entorno y la *observación* accesible al agente, siendo esta última el foco principal del análisis experimental.

El diseño del espacio de observaciones se ha realizado con el objetivo de permitir una modificación progresiva y controlada de la complejidad informacional, manteniendo constantes las dinámicas internas del entorno. De este modo, es posible evaluar cómo distintas representa-

ciones del estado influyen en la velocidad de aprendizaje, la estabilidad del entrenamiento y el rendimiento final del agente.

Formalmente, el entorno se define sobre una rejilla bidimensional discreta. El estado completo incluye, entre otros elementos, la posición del agente, la posición del fantasma, la disposición de muros, la localización de monedas y el estado de posibles comodines. Sin embargo, no toda esta información es necesariamente observable por el agente en todas las configuraciones.

Se han definido varios **modos de observación**, cada uno de los cuales incrementa progresivamente la complejidad del estado observado:

- **Observación mínima (Minimal):** el agente percibe únicamente su propia posición y la posición del fantasma dentro de la rejilla. Esta configuración proporciona una representación muy compacta del estado, pero omite información relevante sobre el entorno, como la localización de las monedas.
- **Observación con comodín binario (Power Boolean):** además de las posiciones del agente y del fantasma, se incluye una variable booleana que indica si un comodín se encuentra activo. Esta extensión introduce información contextual adicional sin aumentar significativamente la dimensionalidad del estado.
- **Observación con temporizador de comodín (Power Timer):** en este modo, la observación incorpora el tiempo restante de activación del comodín. Esta información permite al agente anticipar la duración de su ventaja temporal frente al fantasma, incrementando la complejidad y el carácter temporal del estado.
- **Observación con distribución de monedas (Coins Quadrants):** el entorno se divide en cuadrantes, y la observación incluye información agregada sobre la cantidad de monedas restantes en cada uno de ellos. Este enfoque introduce una representación espacial más rica del entorno sin recurrir a observaciones a nivel de celda.
- **Observación basada en imagen (opcional):** de forma experimental, se contempla un modo de observación en el que el estado se representa como una matriz o imagen que codifica directamente el contenido de cada celda del entorno. Este modo incrementa significativamente la dimensionalidad del estado y permite analizar el comportamiento de los algoritmos ante observaciones de tipo visual.

Cada uno de estos modos define un espacio de observaciones diferente, tanto en dimensionalidad como en contenido semántico. En particular, el aumento progresivo de información no implica necesariamente una mejora del aprendizaje: estados más complejos pueden introducir

redundancia, ruido o dificultar la generalización, especialmente en algoritmos sensibles a la dimensionalidad del espacio de entrada.

Desde el punto de vista experimental, todos los modos de observación se implementan manteniendo invariables el espacio de acciones, la dinámica del entorno y la función de recompensa. Esta decisión de diseño garantiza que cualquier diferencia observada en el rendimiento de los agentes pueda atribuirse principalmente a la complejidad del estado observado y no a otros factores externos.

En los siguientes apartados se describen las dinámicas del entorno, el comportamiento del agente y del enemigo, así como el sistema de recompensas utilizado para guiar el aprendizaje.

3.4. Espacio de acciones y dinámica del agente

El comportamiento del agente en el entorno *SimplePacmanEnv* se define a través de un espacio de acciones discreto y una dinámica determinista, diseñados con el objetivo de simplificar el análisis del aprendizaje y aislar el impacto de la representación del estado.

El espacio de acciones disponible para el agente está compuesto por un conjunto reducido de movimientos discretos, correspondientes a los desplazamientos en las cuatro direcciones cardinales:

- Movimiento hacia arriba.
- Movimiento hacia abajo.
- Movimiento hacia la izquierda.
- Movimiento hacia la derecha.

En cada paso temporal, el agente selecciona una de estas acciones en función de su política actual. La acción se traduce en un intento de desplazamiento dentro de la rejilla del entorno. Si la celda de destino corresponde a un muro, la acción no tiene efecto y el agente permanece en su posición actual. En caso contrario, la posición del agente se actualiza de manera determinista.

Esta dinámica sencilla permite eliminar comportamientos estocásticos no controlados y facilita la interpretación de los resultados experimentales. De este modo, cualquier variación en el rendimiento del agente puede atribuirse principalmente a la política aprendida y a la información observacional disponible, en lugar de a fluctuaciones aleatorias del entorno.

Además del desplazamiento del agente, en cada paso temporal se actualiza el estado interno del entorno, incluyendo la posible recolección de monedas y la interacción con otros elementos, como el fantasma o los comodines. Estas actualizaciones se realizan de manera consistente e independiente

del algoritmo de aprendizaje utilizado, garantizando un comportamiento homogéneo del entorno a lo largo de todos los experimentos.

Desde el punto de vista del aprendizaje, el agente no dispone de conocimiento previo sobre las reglas del entorno ni sobre las consecuencias de sus acciones. La política se construye progresivamente a partir de la experiencia acumulada, guiada únicamente por las recompensas recibidas tras cada transición. Esta configuración refuerza el carácter exploratorio del problema y sitúa el aprendizaje en un marco puramente basado en la interacción agente–entorno.

La simplicidad del espacio de acciones y de la dinámica del agente contribuye a que el foco experimental se sitúe en la complejidad del estado observado, evitando que la dificultad del problema venga dominada por factores externos como acciones continuas, ruido en las transiciones o comportamientos impredecibles del entorno.

3.5. Comportamiento del fantasma

Además del agente controlado por el algoritmo de Aprendizaje por Refuerzo, el entorno *SimplePacmanEnv* incluye un segundo actor: un enemigo denominado *fantasma*. La presencia de este elemento introduce una fuente de riesgo en el entorno, obligando al agente a equilibrar comportamientos exploratorios con estrategias de evitación.

El comportamiento del fantasma se ha diseñado de forma deliberadamente simple y controlada, con el objetivo de no introducir complejidad innecesaria en las dinámicas del entorno. En cada paso temporal, el fantasma ejecuta una acción de movimiento siguiendo una política predefinida, independiente del algoritmo de aprendizaje utilizado por el agente. De este modo, el comportamiento del enemigo es consistente a lo largo de todos los experimentos y no se adapta a la política del agente.

El espacio de acciones del fantasma coincide con el del agente, permitiendo desplazamientos discretos en las cuatro direcciones cardinales. Al igual que en el caso del agente, los movimientos están sujetos a las restricciones impuestas por los muros del entorno, de modo que el fantasma no puede atravesar paredes ni salir de los límites de la rejilla.

Dependiendo de la configuración del entorno, el movimiento del fantasma puede seguir una estrategia determinista simple, como desplazarse hacia la posición del agente, o una política estocástica limitada, seleccionando acciones válidas de manera aleatoria. Esta flexibilidad permite introducir distintos niveles de dificultad sin alterar la estructura general del entorno ni la función de recompensa.

La interacción entre el agente y el fantasma constituye uno de los principales mecanismos de terminación del episodio. Si el agente colisiona con el fantasma en una celda compartida, el episodio finaliza inmediatamente y se aplica una penalización significativa en la recompensa. Este evento incentiva al agente a desarrollar comportamientos evasivos y a planificar sus movimientos

teniendo en cuenta la posición relativa del enemigo.

El diseño del fantasma como un elemento externo con comportamiento fijo permite centrar el análisis experimental en la capacidad del agente para utilizar la información observacional disponible. De este modo, se evita que la complejidad del problema venga dominada por interacciones adversarias dinámicas, manteniendo el foco en la relación entre la representación del estado y el proceso de aprendizaje.

3.6. Sistema de recompensas

El sistema de recompensas constituye el principal mecanismo mediante el cual el agente recibe retroalimentación sobre la calidad de sus acciones y, por tanto, desempeña un papel fundamental en el proceso de aprendizaje. En el entorno *SimplePacmanEnv*, la función de recompensa se ha diseñado cuidadosamente para incentivar comportamientos deseables y penalizar situaciones adversas, manteniendo al mismo tiempo un equilibrio que evite soluciones triviales o inestables.

El objetivo principal del agente es maximizar la recompensa acumulada mediante la recolección de monedas distribuidas en el entorno, al tiempo que evita la colisión con el fantasma. Para ello, el sistema de recompensas se compone de los siguientes elementos:

- **Recompensa por recolección de monedas:** cada vez que el agente recoge una moneda, recibe una recompensa positiva. Este refuerzo incentiva la exploración del entorno y guía al agente hacia las zonas con mayor densidad de recompensas.
- **Penalización por colisión con el fantasma:** si el agente entra en la misma celda que el fantasma, el episodio finaliza y se aplica una penalización negativa significativa. Esta señal refuerza comportamientos evasivos y promueve una planificación más cuidadosa de las acciones.
- **Penalización por paso temporal:** en cada paso del episodio se aplica una pequeña penalización constante. Este término evita que el agente adopte estrategias pasivas o cíclicas, incentivando la resolución eficiente del episodio en el menor número de pasos posible.
- **Recompensa por finalización exitosa del episodio:** cuando el agente logra recolectar todas las monedas del entorno, el episodio finaliza con una recompensa adicional que refuerza el objetivo global de la tarea.

La magnitud relativa de estas recompensas ha sido seleccionada de forma empírica para asegurar un equilibrio adecuado entre exploración, seguridad y eficiencia. Penalizaciones excesivamente severas pueden dificultar el aprendizaje al desalentar la exploración, mientras

que recompensas demasiado elevadas pueden provocar comportamientos miopes centrados en objetivos inmediatos.

Un aspecto relevante del diseño es que la función de recompensa se mantiene idéntica para todas las configuraciones de observación y para todos los algoritmos evaluados. Esta decisión garantiza que cualquier diferencia observada en el rendimiento de los agentes pueda atribuirse a la complejidad del estado observado o al algoritmo de aprendizaje utilizado, y no a variaciones en la señal de refuerzo.

En conjunto, este sistema de recompensas proporciona una señal de aprendizaje clara y coherente con los objetivos del entorno, permitiendo evaluar de forma consistente cómo distintas representaciones del estado influyen en la capacidad del agente para aprender políticas eficaces.

3.7. Condiciones de terminación de los episodios

Las condiciones de terminación determinan cuándo finaliza un episodio de interacción entre el agente y el entorno, y desempeñan un papel relevante tanto en la estabilidad del aprendizaje como en la interpretación de las métricas de evaluación. En el entorno *SimplePacmanEnv*, los episodios se han definido de forma que reflejen situaciones naturales de éxito o fracaso, manteniendo al mismo tiempo una duración acotada.

Un episodio puede finalizar bajo cualquiera de las siguientes condiciones:

- **Finalización exitosa:** el agente ha recolectado todas las monedas presentes en el entorno. Este caso representa el cumplimiento completo del objetivo de la tarea y se acompaña de una recompensa positiva adicional.
- **Colisión con el fantasma:** el agente entra en la misma celda que el fantasma. Esta situación se considera un fallo y provoca la finalización inmediata del episodio con la correspondiente penalización.
- **Límite máximo de pasos:** si el agente supera un número máximo predefinido de pasos sin alcanzar el objetivo, el episodio finaliza automáticamente. Esta condición evita episodios excesivamente largos y comportamientos cíclicos que no conducen a una solución.

El establecimiento de un límite máximo de pasos resulta especialmente relevante para garantizar la comparabilidad entre experimentos. Sin esta restricción, los agentes podrían prolongar indefinidamente los episodios, dificultando la interpretación de métricas como la recompensa acumulada o la duración media de los episodios.

Al igual que ocurre con el resto de elementos del entorno, las condiciones de terminación se mantienen constantes para todas las configuraciones de observación y para todos los algoritmos de

aprendizaje evaluados. De este modo, se asegura que las diferencias observadas en el rendimiento de los agentes se deban a la representación del estado o al algoritmo utilizado, y no a variaciones en la definición del episodio.

En conjunto, estas condiciones de terminación contribuyen a un marco experimental bien definido y controlado, facilitando el análisis comparativo del aprendizaje y la obtención de conclusiones reproducibles.

4. Algoritmos de aprendizaje por refuerzo utilizados

4.1. Configuración común de entrenamiento

Para garantizar la comparabilidad de los resultados obtenidos con los distintos algoritmos de Aprendizaje por Refuerzo (DQN, A2C y PPO), se ha establecido una configuración común de entrenamiento que se aplica de forma uniforme en todos los experimentos. Esta configuración incluye los parámetros generales del entrenamiento, la infraestructura utilizada para la ejecución de los experimentos y el protocolo de evaluación.

Los principales aspectos comunes a todos los experimentos son los siguientes:

- **Entorno de entrenamiento:** todos los algoritmos han sido entrenados en el entorno *SimplePacmanEnv* descrito anteriormente, utilizando el modo de observación seleccionado en cada experimento (ya sea mínimo, con comodín binario, temporizador de poder, o distribución de monedas).
- **Parámetros de entrenamiento:** se ha utilizado un conjunto común de hiperparámetros para los tres algoritmos, con variaciones específicas que se detallarán más adelante para cada algoritmo. Los parámetros incluyen la tasa de aprendizaje, el tamaño del batch, el tamaño del buffer de experiencias y la frecuencia de actualización de la red.
- **Número de pasos de entrenamiento:** el número de pasos de entrenamiento ha sido de 1 millón de pasos por defecto, salvo en algunas configuraciones específicas, como las pruebas de exhaustividad con 2 millones de pasos para A2C. El entrenamiento se ha ejecutado utilizando un criterio de parada temprana, que detiene el entrenamiento si no hay mejoras sustanciales en la política aprendida durante un número definido de evaluaciones consecutivas sin mejora.
- **Evaluación del agente:** para evaluar el rendimiento del agente durante el entrenamiento, se ha implementado un protocolo de evaluación cada 50,000 pasos de entorno, ejecutando múltiples episodios para calcular el rendimiento medio. Además, se ha utilizado una evaluación adicional al final de cada entrenamiento para validar el modelo entrenado.

- **Optimización de hiperparámetros:** inicialmente, los hiperparámetros fueron ajustados manualmente a través de pruebas y errores. Posteriormente, se utilizó la librería Optuna para realizar una búsqueda más sistemática de los mejores valores para parámetros como la tasa de aprendizaje, el tamaño del batch y otros parámetros relevantes para la estabilidad del aprendizaje.

Esta configuración común garantiza que las diferencias observadas entre los algoritmos no sean el resultado de una variabilidad en la configuración del entorno o en los parámetros de entrenamiento, sino que puedan atribuirse a las características específicas de cada algoritmo de Aprendizaje por Refuerzo.

En los siguientes apartados se describirán las configuraciones particulares de cada algoritmo (DQN, A2C y PPO), incluyendo los detalles adicionales que corresponden a cada uno de ellos, como la arquitectura de la red neuronal utilizada, los métodos de exploración y explotación, y los mecanismos de actualización de la política.

4.2. Entrenamiento con Deep Q-Network (DQN)

El algoritmo *Deep Q-Network* (DQN) se ha utilizado en este trabajo como representante de los métodos de Aprendizaje por Refuerzo basados en valor. DQN es especialmente adecuado para entornos con espacios de acción discretos, como el entorno *SimplePacmanEnv*, donde el agente debe seleccionar una acción entre un conjunto finito de movimientos posibles.

En esta implementación, el agente DQN aprende una aproximación de la función de acción-valor $Q(s, a)$ mediante una red neuronal profunda. Dicha red recibe como entrada la observación del estado definida por el modo de observación seleccionado y produce como salida un valor estimado para cada acción disponible. La política seguida durante el entrenamiento es una política ϵ -greedy, que equilibra exploración y explotación permitiendo al agente seleccionar acciones aleatorias con una probabilidad decreciente a lo largo del entrenamiento.

El proceso de entrenamiento se basa en la interacción repetida del agente con el entorno, almacenando las transiciones (s, a, r, s') en un buffer de experiencias. Estas transiciones se muestrean de forma aleatoria para actualizar la red neuronal, reduciendo la correlación temporal entre muestras y mejorando la estabilidad del aprendizaje. Para evitar oscilaciones en la estimación de la función de valor, se emplea una red objetivo (*target network*) que se actualiza de forma periódica con los parámetros de la red principal.

Los principales hiperparámetros utilizados en el entrenamiento con DQN incluyen la tasa de aprendizaje, el tamaño del buffer de experiencias, el tamaño del batch de entrenamiento, el factor de descuento γ y la frecuencia de actualización de la red objetivo. Estos parámetros influyen de manera significativa en la estabilidad y velocidad de convergencia del algoritmo, especialmente

en entornos donde la observación del estado puede variar en complejidad y dimensionalidad.

Debido a la naturaleza secuencial del aprendizaje y al elevado número de interacciones necesarias para que DQN converja hacia una política estable, el coste computacional de este algoritmo resulta considerable en comparación con métodos actor-crítico. Por este motivo, el número de experimentos exhaustivos realizados con DQN fue limitado en relación con A2C y PPO, priorizando configuraciones representativas que permitieran analizar tendencias generales sin incurrir en tiempos de ejecución excesivos.

El rendimiento del agente DQN se evalúa de forma periódica durante el entrenamiento mediante episodios de prueba independientes, lo que permite monitorizar la evolución de la política aprendida y detectar posibles problemas de inestabilidad o sobreajuste. Estas evaluaciones proporcionan una referencia objetiva para comparar el comportamiento de DQN con el de los demás algoritmos considerados en el estudio.

4.3. Entrenamiento con Advantage Actor-Critic (A2C)

El algoritmo *Advantage Actor-Critic* (A2C) se ha empleado en este trabajo como representante de los métodos actor-crítico, que combinan el aprendizaje de una política parametrizada con la estimación del valor de los estados. A diferencia de los métodos basados exclusivamente en valor, como DQN, A2C resulta más flexible ante variaciones en la dimensionalidad del espacio de observación y presenta una mayor estabilidad en entornos con dinámicas complejas.

En A2C, el proceso de aprendizaje se basa en dos componentes principales: un *actor*, encargado de aprender la política $\pi_\theta(a|s)$, y un *crítico*, que estima la función de valor $V_\phi(s)$. El crítico proporciona una señal de error denominada *ventaja* (*advantage*), que mide cuánto mejor resulta ejecutar una acción concreta respecto al valor esperado del estado. Esta señal se utiliza para actualizar la política del actor, reduciendo la varianza del gradiente y mejorando la estabilidad del aprendizaje.

Una de las características distintivas de A2C es el uso de múltiples entornos ejecutados en paralelo durante el entrenamiento. En cada iteración, el agente interactúa simultáneamente con varias instancias del entorno *SimplePacmanEnv*, recopilando trayectorias que se utilizan para actualizar la política de forma síncrona. Este enfoque permite acelerar el aprendizaje y mitigar los efectos negativos de correlación temporal entre muestras consecutivas.

En el contexto de este trabajo, el algoritmo A2C se ha entrenado utilizando arquitecturas de red neuronal compartidas para el actor y el crítico, con capas ocultas densas que procesan la observación del estado en función del modo de observación seleccionado. Los hiperparámetros clave del entrenamiento incluyen la tasa de aprendizaje, el número de pasos por actualización, el coeficiente de entropía y el factor de descuento γ . El coeficiente de entropía desempeña un papel relevante al fomentar la exploración, evitando una convergencia prematura hacia políticas

deterministas subóptimas.

A diferencia de DQN, A2C no requiere una memoria explícita de experiencias ni el uso de redes objetivo, lo que reduce la complejidad de la implementación y el consumo de memoria. Estas características hacen que A2C resulte computacionalmente más eficiente y adecuado para la ejecución de experimentos a gran escala, como la optimización exhaustiva de hiperparámetros realizada en este trabajo.

Debido a su equilibrio entre estabilidad, eficiencia y capacidad de adaptación a distintos espacios de observación, A2C ha sido uno de los algoritmos principales en el estudio experimental. Se han realizado múltiples ejecuciones con distintas configuraciones de hiperparámetros y modos de observación, permitiendo analizar de forma detallada cómo la complejidad del estado influye en el proceso de aprendizaje y en el rendimiento final del agente.

4.4. Entrenamiento con Proximal Policy Optimization (PPO)

El algoritmo *Proximal Policy Optimization* (PPO) ha sido seleccionado en este trabajo debido a su robustez y simplicidad en comparación con otros métodos actor-crítico, como A3C o TRPO (Trust Region Policy Optimization). PPO es una de las técnicas más utilizadas en la práctica de Aprendizaje por Refuerzo Profundo, ya que ofrece un buen equilibrio entre rendimiento y facilidad de implementación.

PPO mejora sobre los métodos actor-crítico tradicionales mediante el uso de una técnica llamada *clipping*, que limita la magnitud de las actualizaciones de la política, evitando cambios drásticos que podrían desestabilizar el entrenamiento. Este método introduce un término adicional en la función de pérdida que penaliza las actualizaciones de la política que se alejan demasiado de la anterior, lo que se expresa de la siguiente manera:

$$L^{CLIP}(\theta) = \mathbb{E} [\text{mín}(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)], \quad (4.1)$$

donde $r_t(\theta)$ es la razón entre la probabilidad de acción de la nueva política y la anterior, y A_t es el valor de la ventaja en el paso temporal t . El parámetro ϵ controla el rango dentro del cual se permite la actualización, y su valor es una constante que equilibra la estabilidad y la capacidad de adaptación de la política.

Una de las principales ventajas de PPO es su capacidad para adaptarse de forma eficiente a diferentes entornos y arquitecturas. A diferencia de TRPO, que requiere cálculos costosos de la matriz Hessiana para cada actualización, PPO emplea una aproximación más simple, basada en el clipping de la probabilidad de acción. Esto reduce significativamente la complejidad computacional, haciendo que PPO sea más fácil de implementar y más rápido de entrenar, lo que lo convierte en un algoritmo ideal para entornos con recursos limitados o experimentos a

gran escala.

El entrenamiento con PPO se ha realizado utilizando una red neuronal de política con arquitectura de capas densas, similar a la utilizada en A2C, con un tamaño de batch mayor y una mayor frecuencia de actualizaciones. El uso de *VecNormalize* para normalizar las observaciones y las recompensas ha sido una de las claves para mejorar la estabilidad y eficiencia del algoritmo, permitiendo que el agente se adapte de manera más rápida y confiable a las distintas configuraciones del entorno.

PPO es conocido por su robustez ante cambios en la complejidad del estado, por lo que se ha seleccionado como uno de los algoritmos principales en este trabajo. Al igual que A2C, PPO ha sido entrenado en paralelo utilizando múltiples instancias del entorno para acelerar el proceso de aprendizaje. Además, como en los otros algoritmos, se ha empleado una estrategia de parada temprana para evitar el sobreajuste y asegurar que el modelo entrenado no se desvíe de una solución óptima.

En resumen, PPO se distingue por su simplicidad, eficiencia y estabilidad, características que lo convierten en el algoritmo ideal para comparar cómo diferentes representaciones del estado afectan al rendimiento del agente. En los siguientes apartados se presentarán los resultados obtenidos con PPO, los cuales servirán como base para las comparaciones entre los distintos algoritmos estudiados en este trabajo.

Capítulo 5

Resultados experimentales

1. Métricas de evaluación y criterios de análisis

La evaluación del rendimiento de los agentes de Aprendizaje por Refuerzo en el entorno *SimplePacmanEnv* se ha realizado utilizando dos métricas principales: la recompensa media por episodio (*mean reward*) y el ratio de completado del entorno (*completion ratio*). Estas métricas capturan aspectos complementarios del comportamiento del agente y permiten un análisis más completo que el uso de una única medida agregada.

La recompensa media por episodio refleja el retorno acumulado obtenido por el agente durante un episodio, teniendo en cuenta tanto las recompensas positivas (recolección de monedas y finalización exitosa) como las penalizaciones (colisiones con el fantasma y coste temporal por paso). Esta métrica es especialmente útil para analizar la eficiencia del comportamiento aprendido, ya que penaliza trayectorias largas o ineficientes incluso cuando el agente logra recolectar una cantidad significativa de monedas.

Por otro lado, el *completion ratio* mide la fracción de monedas recolectadas respecto al total disponible en el entorno durante un episodio. Esta métrica proporciona una medida directa del progreso del agente hacia el objetivo global de la tarea, independientemente de la duración del episodio o del coste temporal asociado. En este sentido, el *completion ratio* permite evaluar hasta qué punto el agente es capaz de explorar el entorno y cubrir sus distintas regiones.

Es importante destacar que ambas métricas capturan comportamientos distintos y no siempre están correlacionadas. Por ejemplo, un agente puede maximizar la recompensa media adoptando una estrategia miope que priorice la recolección de monedas cercanas o seguras, evitando zonas de mayor riesgo. En este caso, el agente puede obtener un *mean reward* elevado sin llegar a completar el entorno, lo que se reflejaría en un *completion ratio* moderado o bajo.

De manera análoga, dos agentes pueden completar sistemáticamente el entorno y, por tanto, presentar valores similares de *completion ratio*, pero diferir en su recompensa media. Esta

situación puede darse cuando uno de los agentes resuelve la tarea de forma más eficiente, empleando menos pasos y evitando penalizaciones temporales innecesarias, lo que se traduce en un mayor *mean reward*. En contraste, un agente que complete el entorno de forma más lenta o errática obtendrá una recompensa media inferior a pesar de alcanzar el mismo grado de completado.

Por este motivo, el análisis experimental presentado en este capítulo se apoya en ambas métricas de manera complementaria. El *mean reward* se utiliza para evaluar la eficiencia y estabilidad del comportamiento aprendido, mientras que el *completion ratio* permite medir el grado de cumplimiento del objetivo global de la tarea. La interpretación conjunta de ambas métricas resulta esencial para comprender adecuadamente el impacto de la complejidad del estado en el proceso de aprendizaje.

2. Resultados por algoritmo

En este capítulo se presentan los resultados obtenidos a partir de los experimentos descritos en el capítulo anterior. El objetivo es analizar el comportamiento de los distintos algoritmos de Aprendizaje por Refuerzo evaluados —DQN, A2C y PPO— bajo diferentes configuraciones de observación, manteniendo constantes las dinámicas del entorno y el sistema de recompensas.

Los resultados se han obtenido a partir de múltiples ejecuciones independientes, utilizando diferentes semillas aleatorias con el fin de reducir la variabilidad inherente al proceso de aprendizaje. Para cada configuración se han registrado métricas cuantitativas como la recompensa media por episodio, la duración media de los episodios y la estabilidad del entrenamiento, así como métricas derivadas de la evaluación periódica de los modelos entrenados.

Este apartado se centra en describir el comportamiento de cada algoritmo de forma individual, sin establecer todavía comparaciones directas entre ellos. El análisis comparativo se abordará en secciones posteriores, una vez presentados los resultados de cada método de manera aislada.

2.1. Resultados con Deep Q-Network (DQN)

En este apartado se presentan los resultados obtenidos con el algoritmo Deep Q-Network (DQN) bajo las distintas configuraciones de observación definidas en el entorno *SimplePacmanEnv*. El objetivo es analizar el comportamiento del algoritmo de forma aislada, sin establecer aún comparaciones directas con otros métodos de aprendizaje por refuerzo.

Tras una fase inicial de ajuste manual, los hiperparámetros de DQN fueron optimizados mediante la librería Optuna utilizando 40 *trials* y 300 000 pasos de entrenamiento por prueba para cada modo de observación. Posteriormente, los modelos seleccionados fueron entrenados de manera más exhaustiva durante 5 millones de pasos, aplicando un protocolo de evaluación

periódica con 75 episodios, una frecuencia de evaluación de 20 000 pasos y un criterio de parada temprana. Finalmente, cada modelo fue evaluado sobre 100 episodios independientes para obtener métricas finales de rendimiento.

Los resultados muestran que DQN es capaz de aprender comportamientos parcialmente efectivos en el entorno, con un rendimiento muy dependiente del modo de observación. En particular, el modo *coins_quadrants* presenta la recompensa media más elevada (41.81), seguido por *power_time* (37.18) y *bool_power* (31.22). En contraste, el modo de observación mínima obtiene una recompensa media negativa (-14.56), lo que sugiere que una representación excesivamente simplificada dificulta el aprendizaje de políticas útiles en este entorno.

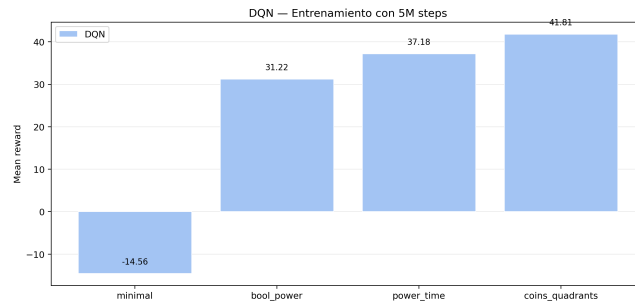


Figura 5.1: Mean reward con algoritmo de DQN entrenado con 5M de steps

No obstante, a pesar de estas mejoras en la recompensa acumulada, DQN no consigue completar el entorno de forma consistente en ninguno de los modos de observación analizados. La tasa de éxito es nula en todos los casos, y no se observan episodios cercanos a la resolución completa del nivel. Este comportamiento se refleja también en el *completion ratio*, que alcanza su valor máximo en el modo *coins_quadrants* (0.68), seguido por *power_time* (0.61) y *bool_power* (0.53). El modo de observación mínima presenta el menor ratio de completado (0.47), lo que refuerza la idea de que la falta de información espacial limita la capacidad del agente para cubrir el entorno de manera efectiva. En todos los casos, la tasa de éxito permanece nula, lo que indica que el agente no logra completar el nivel de forma consistente.

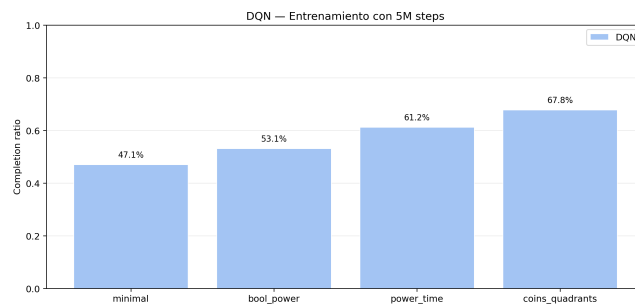


Figura 5.2: Completion ratio con algoritmo de DQN entrenado con 5M de steps

En términos de estabilidad, los resultados evidencian una elevada variabilidad en el rendimiento entre episodios, reflejada en desviaciones estándar de la recompensa relativamente altas, especialmente en los modos con observaciones más complejas. Asimismo, la duración media de los episodios es considerablemente elevada, lo que sugiere comportamientos poco eficientes y trayectorias prolongadas que no conducen de forma sistemática a la finalización exitosa del episodio.

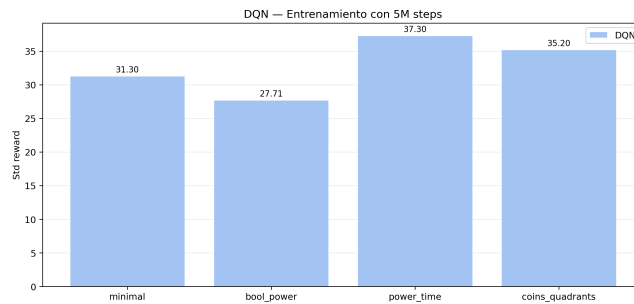


Figura 5.3: Std reward con algoritmo de DQN entrenado con 5M de steps

El análisis de los distintos modos de observación indica que, en el caso de DQN, el aumento de la complejidad del estado se traduce en una mejora progresiva del rendimiento, tanto en términos de recompensa media como de *completion ratio*. A medida que se incorporan observaciones más informativas, el agente es capaz de explotar mejor la estructura del entorno y cubrir una mayor fracción del mapa. No obstante, esta mejora no se refleja en una tasa de éxito no nula, lo que pone de manifiesto que, pese al mayor progreso parcial, el agente no logra resolver la tarea de forma consistente.

Finalmente, cabe destacar el elevado coste computacional asociado al entrenamiento de DQN. El número de pasos necesarios para alcanzar un comportamiento razonablemente estable, junto con la necesidad de múltiples ejecuciones para el ajuste de hiperparámetros, limita la viabilidad de realizar estudios exhaustivos con este algoritmo en comparación con métodos actor-crítico más eficientes.

En conjunto, estos resultados posicionan a DQN como un algoritmo capaz de aprender comportamientos parciales en el entorno *SimplePacmanEnv*, pero con limitaciones claras en términos de estabilidad, eficiencia y capacidad de resolución completa de la tarea.

2.2. Resultados con Advantage Actor-Critic (A2C)

En este apartado se presentan los resultados obtenidos con el algoritmo Advantage Actor-Critic (A2C) bajo las distintas configuraciones de observación definidas en el entorno *SimplePacmanEnv*. A diferencia de DQN, A2C permitió realizar un mayor número de ejecuciones

exhaustivas debido a su menor coste computacional, lo que facilitó un análisis más detallado del impacto de los hiperparámetros y de la complejidad del estado.

Se evaluaron tres configuraciones principales de entrenamiento, correspondientes a diferentes niveles de optimización de hiperparámetros mediante Optuna: (i) 40 *trials* con 300 000 pasos por prueba, (ii) 40 *trials* con 1 millón de pasos y (iii) 200 *trials* con 1 millón de pasos. En todos los casos, los modelos resultantes fueron entrenados durante 5 millones de pasos y evaluados posteriormente sobre 100 episodios independientes.

Los resultados muestran que A2C es capaz de aprender políticas estables y consistentes en todas las configuraciones de observación analizadas. En comparación con DQN, el algoritmo presenta una menor variabilidad en la recompensa entre episodios y una duración media de los episodios significativamente más reducida, lo que indica un comportamiento más estructurado y menos errático. No obstante, al igual que en el caso de DQN, A2C no logra completar el entorno de forma exitosa en ninguno de los modos de observación evaluados, con tasas de éxito y episodios cercanos a la resolución completas nulas.

El análisis de la complejidad del estado revela diferencias claras entre los distintos modos de observación. Las configuraciones que incorporan información contextual adicional, como el temporizador del comodín (*power_time*), presentan de forma consistente mayores ratios de completado parcial del entorno, alcanzando valores cercanos al 18 %. Por el contrario, el modo de observación mínima obtiene los peores resultados, con ratios de completado notablemente inferiores, lo que sugiere que una representación excesivamente simple limita la capacidad del agente para desarrollar estrategias efectivas.

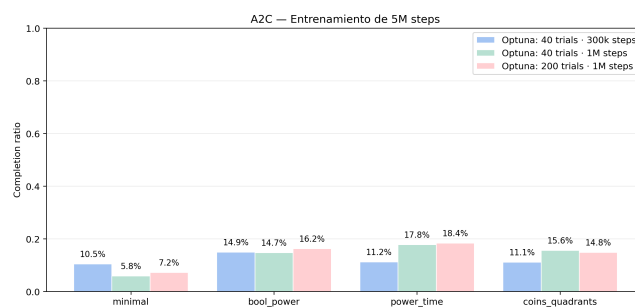


Figura 5.4: Completion ratio con algoritmo de A2C entrenado con 5M de steps

En cuanto al impacto de la optimización de hiperparámetros, los resultados indican que el aumento del esfuerzo computacional no se traduce necesariamente en mejoras sustanciales del rendimiento. Las configuraciones más exhaustivas, con un mayor número de *trials* y pasos de entrenamiento, no presentan diferencias significativas respecto a configuraciones más ligeras, lo que apunta a un fenómeno de rendimientos decrecientes en el proceso de ajuste de A2C para este entorno.

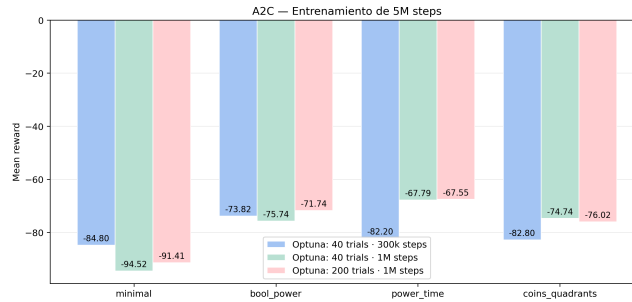


Figura 5.5: Completion ratio con algoritmo de A2C entrenado con 5M de steps

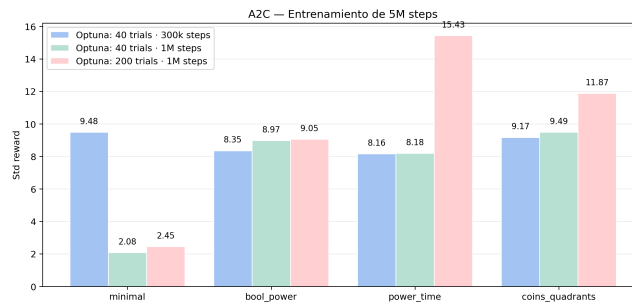


Figura 5.6: Std reward con algoritmo de A2C entrenado con 5M de steps

En conjunto, estos resultados muestran que A2C ofrece una mayor estabilidad que DQN y se beneficia de observaciones más informativas, pero sigue mostrando limitaciones claras para resolver completamente la tarea. Este comportamiento refuerza la idea de que la complejidad del estado, si bien necesaria para aprender políticas útiles, no garantiza por sí sola el éxito del aprendizaje en entornos con dinámicas adversas.

2.3. Resultados con Proximal Policy Optimization (PPO)

En este apartado se presentan los resultados obtenidos con el algoritmo Proximal Policy Optimization (PPO) bajo las distintas configuraciones de observación del entorno *SimplePacmanEnv*. PPO se ha evaluado utilizando dos niveles de optimización de hiperparámetros mediante Optuna, correspondientes a una configuración ligera (40 *trials* con 300 000 pasos por prueba) y una configuración exhaustiva (200 *trials* con 1 millón de pasos por prueba). En ambos casos, los modelos seleccionados se entrenaron posteriormente durante 5 millones de pasos antes de su evaluación final.

Los resultados evidencian una marcada sensibilidad de PPO al grado de optimización de hiperparámetros. Las configuraciones menos exhaustivas presentan un rendimiento limitado, con recompensas medias negativas y bajos ratios de completado parcial del entorno. En contraste, los modelos entrenados a partir de una optimización más exhaustiva muestran una mejora

sustancial en todas las métricas consideradas, lo que pone de manifiesto la importancia del ajuste fino de los parámetros en este algoritmo.

Bajo la configuración más exhaustiva, PPO obtiene resultados competitivos, especialmente en configuraciones de observación más simples, como el modo *minimal*. Sin embargo, en modos de observación más ricos, DQN alcanza valores superiores tanto de recompensa media como de *completion ratio*. Estos resultados indican que, si bien PPO es capaz de aprender políticas efectivas cuando se dispone de una optimización adecuada, no domina de forma sistemática frente a DQN en este entorno.

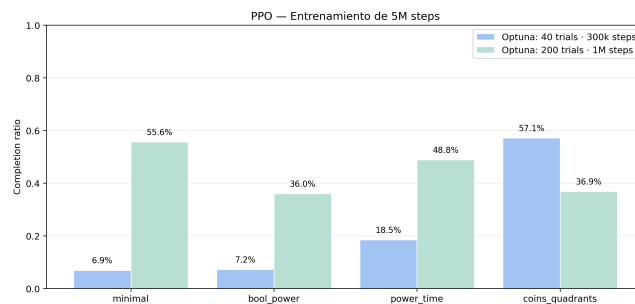


Figura 5.7: Completion ratio con algoritmo de PPO entrenado con 5M de steps

No obstante, a pesar de estas mejoras, PPO no logra resolver completamente el entorno en ninguno de los modos de observación evaluados, manteniendo tasas de éxito nulas en todas las configuraciones. Asimismo, se observa una elevada variabilidad en la recompensa entre episodios y una duración media de los episodios considerablemente elevada, especialmente en los modelos mejor optimizados. Este comportamiento indica que, si bien las políticas aprendidas son más ambiciosas y exploratorias, aún carecen de la consistencia necesaria para completar la tarea de forma fiable.

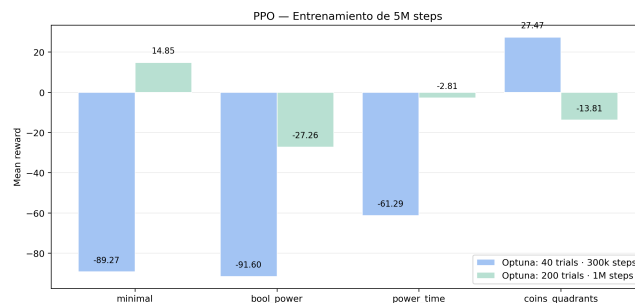


Figura 5.8: Mean reward con algoritmo de PPO entrenado con 5M de steps

El análisis de los distintos modos de observación revela que PPO se beneficia especialmente de representaciones del estado que incorporan información estructurada del entorno, como la distribución de monedas por cuadrantes. Sin embargo, el aumento de la complejidad del estado

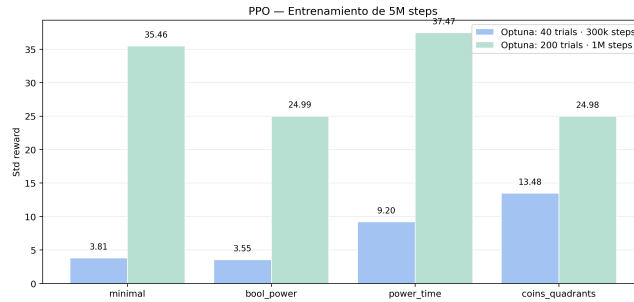


Figura 5.9: Std reward con algoritmo de PPO entrenado con 5M de steps

no garantiza una mejora sistemática del rendimiento, lo que refuerza la hipótesis de que existe un equilibrio óptimo entre simplicidad y riqueza informacional.

En conjunto, estos resultados posicionan a PPO como el algoritmo con mayor potencial en el entorno *SimplePacmanEnv*, siempre que se disponga de un ajuste de hiperparámetros adecuado. Su capacidad para aprovechar observaciones más complejas y su rendimiento superior respecto a DQN y A2C lo convierten en un candidato especialmente relevante para el análisis comparativo presentado en las siguientes secciones.

3. Comparación entre algoritmos

En este apartado se comparan los resultados obtenidos con los tres algoritmos de Aprendizaje por Refuerzo evaluados en este trabajo: DQN, A2C y PPO. El objetivo es analizar sus diferencias en términos de rendimiento, estabilidad y eficiencia, manteniendo constantes las dinámicas del entorno, la función de recompensa y el protocolo de evaluación.

La comparación se realiza a partir de los mejores modelos obtenidos para cada algoritmo, seleccionados tras los correspondientes procesos de optimización de hiperparámetros y entrenados durante un número equivalente de pasos de entorno. De este modo, se garantiza que las diferencias observadas reflejen principalmente las características inherentes a cada algoritmo y su interacción con la complejidad del estado observado.

Desde el punto de vista del rendimiento absoluto, los resultados muestran que DQN es el algoritmo que alcanza los mayores valores tanto de recompensa media como de *completion ratio*, especialmente a medida que aumenta la complejidad de la observación. PPO obtiene resultados competitivos en configuraciones concretas y muestra una elevada capacidad de adaptación cuando se dispone de una optimización exhaustiva de hiperparámetros, mientras que A2C presenta un comportamiento más estable pero claramente inferior en términos de progreso hacia la resolución del entorno. No obstante, ninguno de los algoritmos logra completar el entorno de manera sistemática, manteniendo tasas de éxito nulas en todas las configuraciones evaluadas.

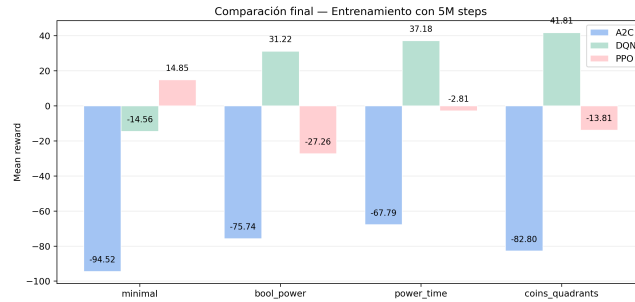


Figura 5.10: Comparativa de mean reward entre algoritmos con 5M de steps

En términos de estabilidad del aprendizaje, se observan diferencias claras entre los algoritmos. DQN presenta una elevada variabilidad entre episodios, con grandes oscilaciones en la recompensa y episodios de duración prolongada, lo que indica políticas poco consistentes y sensibles a la configuración del estado. A2C, por el contrario, muestra un comportamiento más estable, con desviaciones estándar de recompensa menores y episodios más cortos, aunque a costa de un rendimiento global inferior. PPO se sitúa en un punto intermedio: logra recompensas y ratios de completado superiores, pero con una variabilidad notable, especialmente en configuraciones de observación más complejas.

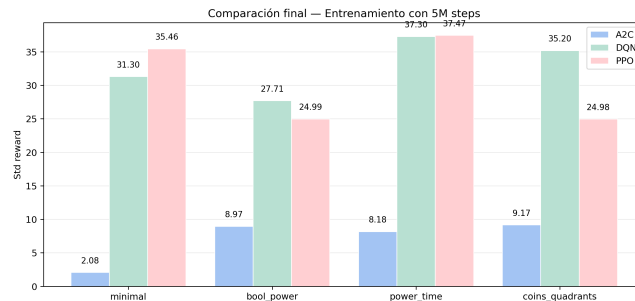


Figura 5.11: Comparativa de std reward entre algoritmos con 5M de steps

El coste computacional constituye otro factor diferenciador relevante. DQN resulta el algoritmo más costoso en términos de tiempo de entrenamiento y recursos necesarios para alcanzar un comportamiento razonablemente estable, lo que limita la viabilidad de realizar estudios exhaustivos de optimización. A2C presenta una mayor eficiencia computacional, permitiendo ejecutar un mayor número de experimentos y configuraciones en tiempos razonables. PPO, aunque más costoso que A2C, ofrece una mejor relación entre esfuerzo computacional y rendimiento final cuando se dispone de una optimización de hiperparámetros adecuada.

Finalmente, la sensibilidad a la complejidad del estado difiere de forma significativa entre algoritmos. DQN muestra un deterioro progresivo del rendimiento a medida que aumenta la dimensionalidad del espacio de observación, mientras que A2C tolera mejor observaciones más

ricas, aunque sin traducir esta información adicional en mejoras sustanciales del rendimiento. PPO es el algoritmo que mejor aprovecha representaciones del estado más estructuradas, como la distribución de monedas por cuadrantes, aunque incluso en este caso el aumento de complejidad no garantiza una mejora sistemática.

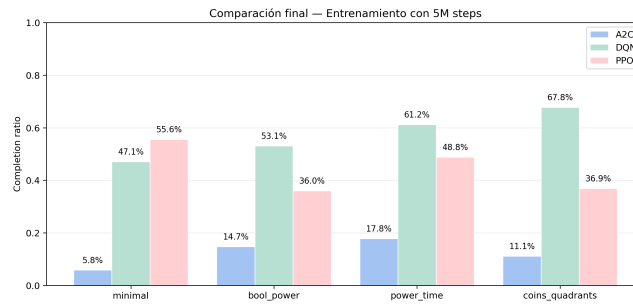


Figura 5.12: Comparativa de completion ratio entre algoritmos con 5M de steps

En conjunto, estos resultados evidencian que no existe un algoritmo universalmente superior, sino que el rendimiento depende tanto del método de aprendizaje como de la representación del estado. PPO emerge como el algoritmo más prometedor para el entorno estudiado, mientras que DQN y A2C presentan limitaciones claras en términos de estabilidad y capacidad de explotación de la información observacional.

4. Impacto de la complejidad del estado

El objetivo principal de este trabajo es analizar cómo la complejidad de la representación del estado influye en el rendimiento de los algoritmos de Aprendizaje por Refuerzo. En este apartado se integran los resultados obtenidos con DQN, A2C y PPO para evaluar de forma transversal el efecto de los distintos modos de observación definidos en el entorno *SimplePacmanEnv*.

Para ello, se han considerado cuatro configuraciones principales de observación: observación mínima, observación con comodín binario, observación con temporizador de comodín y observación con distribución de monedas por cuadrantes. Estas configuraciones permiten incrementar progresivamente la cantidad y riqueza de la información accesible al agente, manteniendo constantes las dinámicas del entorno y el sistema de recompensas.

Los resultados muestran que la relación entre complejidad del estado y rendimiento del agente depende del algoritmo considerado. En particular, DQN presenta una tendencia clara de mejora progresiva del rendimiento a medida que se incorporan observaciones más informativas. Sin embargo, esta relación no se mantiene de forma uniforme en todos los algoritmos, lo que indica que el beneficio de estados más complejos está condicionado por la capacidad del método de aprendizaje para explotar dicha información adicional.

En los subapartados siguientes se analiza el comportamiento observado para cada tipo de representación del estado, integrando los resultados de los distintos algoritmos evaluados.

4.0.1. Observación mínima

La observación mínima proporciona al agente únicamente información básica sobre la posición del propio agente y del fantasma, omitiendo cualquier detalle sobre la distribución de monedas o el estado de los comodines. Esta configuración representa el nivel más bajo de complejidad del estado evaluado en el estudio.

En esta modalidad, los tres algoritmos muestran limitaciones claras en su capacidad de aprendizaje. DQN presenta recompensas medias bajas y un comportamiento altamente inestable, mientras que A2C obtiene los peores ratios de completado parcial del entorno entre todas las configuraciones evaluadas. PPO, aunque logra mejorar de forma notable el rendimiento cuando se optimiza de manera exhaustiva, tampoco alcanza una resolución consistente del entorno.

Estos resultados sugieren que una representación del estado excesivamente simplificada no proporciona la información necesaria para que el agente pueda planificar estrategias efectivas a largo plazo. En ausencia de información sobre la localización de objetivos, el aprendizaje se basa en comportamientos reactivos y locales, lo que limita la capacidad de completar la tarea.

4.0.2. Observación con comodín binario

La incorporación de una variable booleana que indica la activación de un comodín introduce información contextual adicional sin aumentar significativamente la dimensionalidad del estado. Este modo de observación permite al agente diferenciar entre situaciones de riesgo y ventaja temporal frente al fantasma.

Los resultados muestran que esta información adicional mejora el rendimiento respecto a la observación mínima en algunos algoritmos, especialmente en DQN y PPO, donde se observan incrementos en la recompensa media y en el ratio de completado parcial del entorno. Sin embargo, el impacto de esta mejora es limitado y no se traduce en una mayor tasa de éxito.

En el caso de A2C, la incorporación del comodín binario no produce mejoras significativas, lo que sugiere que este algoritmo requiere información más estructurada o rica para aprovechar de forma efectiva el contexto adicional. En conjunto, este modo de observación representa una mejora marginal, pero insuficiente para resolver las limitaciones estructurales del aprendizaje.

4.0.3. Observación con temporizador de comodín

El modo de observación que incluye el temporizador del comodín incrementa la complejidad del estado al introducir una dimensión temporal explícita. Esta información permite al agente

anticipar la duración de su ventaja frente al fantasma y potencialmente planificar trayectorias más arriesgadas de forma controlada.

Este modo de observación muestra mejoras consistentes en A2C y PPO respecto a configuraciones más simples, alcanzando los mayores ratios de completado parcial del entorno en el caso de A2C. Sin embargo, el aumento de la complejidad también se acompaña de una mayor variabilidad en el rendimiento, especialmente en PPO, donde se observan episodios largos y comportamientos menos consistentes.

Estos resultados indican que la introducción de información temporal puede ser beneficiosa, pero también incrementa la dificultad del aprendizaje. La capacidad del algoritmo para explotar esta información depende en gran medida de su estabilidad y de la calidad del ajuste de hiperparámetros.

4.0.4. Observación con distribución de monedas por cuadrantes

La representación del estado mediante la distribución de monedas por cuadrantes proporciona una visión estructurada del entorno, capturando información espacial relevante sin recurrir a observaciones a nivel de celda. Este enfoque busca un equilibrio entre riqueza informacional y control de la dimensionalidad.

Los resultados experimentales muestran que este modo de observación es uno de los más efectivos, especialmente en el caso de PPO, donde se alcanzan los mayores ratios de completado parcial del entorno. Esta representación permite al agente inferir regiones de interés y planificar desplazamientos más coherentes, mejorando la eficiencia del aprendizaje.

No obstante, incluso en esta configuración, ninguno de los algoritmos logra completar el entorno de forma sistemática. Esto sugiere que, aunque la información espacial agregada resulta beneficiosa, sigue existiendo un límite en la capacidad de los algoritmos evaluados para integrar y explotar dicha información en un entorno con dinámicas adversas.

Capítulo 6

Conclusiones y trabajo futuro

1. Conclusiones del trabajo

El presente trabajo ha analizado de forma empírica el impacto de la complejidad de la representación del estado en el rendimiento de distintos algoritmos de Aprendizaje por Refuerzo en un entorno controlado inspirado en el videojuego Pac-Man. A través del diseño de un entorno propio y de una batería de experimentos sistemáticos, se ha podido estudiar cómo diferentes configuraciones de observación influyen en la estabilidad, eficiencia y calidad de las políticas aprendidas.

Los resultados obtenidos muestran que la relación entre complejidad del estado y rendimiento del agente depende fuertemente del algoritmo considerado. Representaciones excesivamente simples limitan la capacidad del agente para planificar estrategias efectivas, mientras que el aumento de la complejidad puede traducirse en mejoras sustanciales cuando el algoritmo es capaz de explotar la información adicional. No obstante, dicho aumento también puede incrementar la varianza del aprendizaje y la sensibilidad a la configuración del entrenamiento, por lo que su beneficio no es universal ni garantiza la resolución completa de la tarea.

Entre los algoritmos analizados, DQN ha alcanzado los mayores valores de recompensa media y de *completion ratio*, especialmente al incorporar observaciones más informativas, lo que sugiere una alta capacidad para explotar representaciones del estado estructuradas en este entorno. PPO ha mostrado un comportamiento competitivo en configuraciones concretas y una elevada capacidad de mejora cuando se dispone de una optimización exhaustiva de hiperparámetros, mientras que A2C ha destacado por su estabilidad pero con un rendimiento global inferior en términos de progreso hacia el objetivo. No obstante, ninguno de los algoritmos ha logrado resolver completamente la tarea de manera consistente, manteniendo tasas de éxito nulas, lo que pone de manifiesto la dificultad inherente del problema y las limitaciones de los enfoques considerados.

En conjunto, el trabajo confirma que el diseño del espacio de observación es un factor determinante en el éxito del aprendizaje por refuerzo, y que dicho diseño debe equilibrar cuidadosamente simplicidad y expresividad en función del algoritmo y del entorno considerado.

2. Evaluación del grado de logro de los objetivos

Los objetivos planteados al inicio del proyecto se han alcanzado de forma satisfactoria. En primer lugar, se ha diseñado e implementado un entorno de Aprendizaje por Refuerzo propio, compatible con el ecosistema Gymnasium y Stable-Baselines3, que permite modificar de manera controlada la complejidad de la observación sin alterar las dinámicas ni la función de recompensa.

En segundo lugar, se han entrenado y evaluado distintos algoritmos representativos del Aprendizaje por Refuerzo Profundo bajo múltiples configuraciones de estado, utilizando un protocolo experimental homogéneo y reproducible. Este enfoque ha permitido analizar de forma sistemática el impacto de la representación del estado sobre el rendimiento del aprendizaje.

Finalmente, se ha llevado a cabo un análisis comparativo detallado que ha permitido extraer conclusiones fundamentadas sobre las ventajas y limitaciones de cada algoritmo y sobre el papel de la complejidad observacional. Aunque el objetivo de resolver completamente el entorno no se ha alcanzado, este hecho constituye en sí mismo un resultado relevante que refuerza las conclusiones extraídas.

3. Evaluación crítica de la planificación y metodología

Desde el punto de vista metodológico, el proyecto ha seguido un enfoque riguroso y estructurado, apoyado en buenas prácticas de desarrollo de software y experimentación en aprendizaje automático. La modularidad del código, el uso de un repositorio versionado y la adopción de herramientas estándar como Optuna y TensorBoard han facilitado la reproducibilidad y el análisis sistemático de los resultados.

No obstante, el proyecto también ha puesto de manifiesto algunas limitaciones prácticas. El elevado coste computacional asociado al entrenamiento y ajuste de hiperparámetros, especialmente en algoritmos como DQN y PPO, ha condicionado el número de configuraciones y ejecuciones exhaustivas que podían realizarse. Asimismo, la dependencia de los resultados respecto a la semilla aleatoria introduce una variabilidad que, aunque mitigada mediante evaluaciones múltiples, no puede eliminarse por completo.

En retrospectiva, una planificación inicial que incorporase desde etapas tempranas herramientas de optimización automática de hiperparámetros podría haber reducido el tiempo dedicado a ajustes manuales. A pesar de ello, la metodología empleada ha resultado adecuada para

responder a la pregunta de investigación planteada.

4. Consideraciones de sostenibilidad, diversidad y aspectos ético-sociales

El proyecto presenta un impacto limitado en términos de sostenibilidad, diversidad y aspectos ético-sociales, dado su carácter fundamentalmente técnico y experimental. No obstante, el elevado consumo computacional asociado al entrenamiento de modelos de Aprendizaje por Refuerzo plantea consideraciones relevantes en términos de eficiencia energética y huella de carbono, especialmente en contextos de investigación a gran escala.

Desde una perspectiva ética, el trabajo no involucra datos personales ni aplicaciones con impacto directo sobre individuos o colectivos, por lo que no se identifican riesgos significativos en términos de privacidad, sesgo o discriminación. Sin embargo, los resultados obtenidos subrayan la importancia de diseñar agentes de aprendizaje con representaciones del estado adecuadas, ya que decisiones erróneas en este ámbito pueden dar lugar a comportamientos subóptimos o impredecibles en aplicaciones reales.

5. Trabajo futuro

A partir de los resultados obtenidos, se identifican diversas líneas de trabajo futuro que podrían ampliar y profundizar el análisis realizado. En primer lugar, sería de interés explorar representaciones del estado basadas en observaciones visuales completas, utilizando arquitecturas convolucionales que permitan aprender directamente a partir de imágenes del entorno.

Asimismo, la introducción de mecanismos de memoria, como redes recurrentes o arquitecturas basadas en atención, podría mejorar el desempeño en entornos parcialmente observables y permitir una mejor explotación de información temporal. Otra línea prometedora consiste en analizar la capacidad de generalización de los agentes entrenados, evaluando su comportamiento en mapas o configuraciones del entorno no vistas durante el entrenamiento.

Finalmente, futuros trabajos podrían abordar el estudio de técnicas de aprendizaje por refuerzo multiagente o la incorporación de estrategias de aprendizaje jerárquico, con el objetivo de descomponer la tarea en subobjetivos y facilitar la planificación a largo plazo. Estas extensiones permitirían profundizar en la relación entre percepción, complejidad del estado y toma de decisiones en entornos más realistas.

Bibliografia

- Maxime Chevalier-Boisvert, David Willems, and Suman Pal. Minimalistic gridworld environment for openai gym. In *GitHub repository: <https://github.com/maximecb/gym-minigrid>*, 2018.
- Karl Cobbe, Oleg Klimov, Chris Hesse, Tae Kim, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, pages 2048–2056, 2020.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2020.
- Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *Proceedings of AAAI Fall Symposium Series*, 2015.
- Timothée Lesort, Mathieu Seurin, Natalia Díaz-Rodríguez, and David Filliat. State representation learning for control: An overview. *arXiv preprint arXiv:1802.04181*, 2018.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dhharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937, 2016.
- OpenAI. Gymnasium documentation, 2023. URL <https://gymnasium.farama.org/>.
- Antonin Raffin, Ashley Hill, Maximilian Ernestus, Adam Gleave, Anssi Kanervisto, Noah Dormann, et al. Stable-baselines3: Reliable reinforcement learning implementations. *Journal*

of Machine Learning Research, 22(268):1–8, 2021. URL <https://www.jmlr.org/papers/v22/20-1364.html>.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018. URL <http://incompleteideas.net/book/the-book-2nd.html>.